

figures/escudo_UPV_vertical.png

UNIVERSIDAD POLITÉCNICA DE VALENCIA

Escuela Técnica Superior de Ingeniería Informática

Departamento de Sistemas Informáticos y Computación

Trabajo Fin de Máster

Arquitectura multiagente para la toma de decisiones en mercados de activos digitales

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas
e Imagen Digital

Autor

Shiyi Cheng

Tutora

Eva Onaindia de la Rivaherrera

Curso 2025–2026

Resumen

Los mercados de activos digitales mueven cientos de millones anuales y, aun así, carecen de herramientas analíticas y de automatizaciones integradas o de libre uso. La motivación principal de este proyecto nace de esta profunda brecha tecnológica. Actualmente, la gestión de carteras en la mayoría de los mercados se realiza de forma manual o mediante herramientas estáticas, lo que resulta altamente ineficiente y arriesgado.

El propósito principal de este TFM es evaluar si un ecosistema descentralizado de agentes de inteligencia artificial puede superar las limitaciones humanas y generar rendimientos estables en mercados de alta fricción. Se opta por una arquitectura multiagente de aprendizaje por refuerzo para descentralizar la toma de decisiones y reducir la complejidad computacional del mercado. El objetivo es que los agentes aprendan estrategias de inversión óptimas que maximicen el saldo total de la cartera.

La metodología desarrollada en el TFM se aplica al ámbito de la economía virtual de Counter-Strike 2, un popular videojuego que cuenta con un ecosistema financiero integrado de alta frecuencia y liquidez.

Palabras clave: Mercados de activos digitales, toma de decisiones, arquitectura multiagente, aprendizaje por refuerzo.

Resum

Els mercats d'actius digitals mouen centenars de milions anuals i, tot i això, encara no disposen d'eines analítiques ni d'automatitzacions integrades o de lliure ús. La motivació principal d'aquest projecte naix d'aquesta profunda bretxa tecnològica. Actualment, la gestió de carteres en la majoria dels mercats es realitza de forma manual o mitjançant eines estàtiques, la qual cosa resulta altament ineficient i arriscada.

El propòsit principal d'aquest TFM és avaluar si un ecosistema descentralitzat d'agents d'intel·ligència artificial pot superar les limitacions humanes i generar rendiments estables en mercats d'alta fricció. S'opta per una arquitectura multiagent d'aprenentatge per reforç per descentralitzar la presa de decisions i reduir la complexitat computacional del mercat. L'objectiu és que els agents aprenguen estratègies d'inversió òptimes que maximitzen el saldo total de la cartera.

La metodologia desenvolupada en el TFM s'aplica a l'àmbit de l'economia virtual de Counter-Strike 2, un videojoc popular que compta amb un ecosistema financer integrat d'alta freqüència i liquiditat.

Paraules clau: Mercats d'actius digitals, presa de decisions, arquitectura multiagent, aprenentatge per reforç.

Abstract

Digital asset markets move hundreds of millions annually and yet still lack integrated or freely available analytical tools and automation systems. The main motivation of this project stems from this deep technological gap. At present, portfolio management in most markets is carried out manually or through static tools, which makes it highly inefficient and risky.

The main purpose of this master's thesis is to evaluate whether a decentralized ecosystem of artificial intelligence agents can overcome human limitations and generate stable returns in high-friction markets. A multi-agent reinforcement learning architecture is chosen to decentralize decision-making and reduce the computational complexity of the market. The goal is for the agents to learn optimal investment strategies that maximize the total portfolio balance.

The methodology developed in this master's thesis is applied to the virtual economy of Counter-Strike 2, a popular video game with an integrated financial ecosystem characterized by high frequency and liquidity.

Key words: Digital asset markets, decision-making, multi-agent architecture, reinforcement learning.

Índice general

Índice de figuras	7
Índice de tablas	8
Lista de algoritmos	9
Siglas y abreviaturas	10
1 Introducción	12
1.1 Motivación	12
1.2 Planteamiento del problema	13
1.3 Objetivos	14
1.4 Alcance y restricciones	14
1.5 Metodología de desarrollo	15
1.6 Estructura del documento	15
2 Marco teórico y estado del arte	17
2.1 Mercados de activos digitales	17
2.1.1 Activos virtuales en Counter-Strike 2	17
2.1.2 Fricciones de mercado	18
2.2 Aprendizaje por refuerzo	18
2.2.1 Procesos de decisión de Markov	18
2.2.2 Política, retorno y exploración	19
2.3 Aprendizaje por refuerzo multiagente	19
2.3.1 Cooperación y coordinación entre agentes	19
2.3.2 Entrenamiento centralizado y ejecución descentralizada	20
2.4 Toma de decisiones en carteras	21
2.4.1 Rentabilidad y riesgo	21
2.4.2 Liquidez y costes de transacción	21
2.5 Aplicaciones relacionadas	21
2.6 Tecnologías utilizadas	22
2.6.1 Herramientas de desarrollo	22
2.6.2 Adquisición y tratamiento de datos	22
2.6.3 Persistencia y modelo de datos	23
2.6.4 Predicción, simulación y entrenamiento multiagente	23
2.6.5 Interfaz y operación local	23
3 Arquitectura multiagente propuesta	24
3.1 Formulación técnica del sistema	24
3.2 Visión general del sistema	24
3.3 Flujo operativo	25
3.4 Definición del entorno	25
3.4.1 Estado y observaciones	26
3.4.2 Espacio de acciones	26
3.4.3 Transición del entorno	27
3.5 Agentes del sistema	27

3.5.1	Scout	27
3.5.2	Trader	27
3.5.3	Portfolio	27
3.6	Función de recompensa	28
3.6.1	Recompensa individual	28
3.6.2	Recompensa cooperativa	28
3.7	Restricciones de riesgo	28
4	Configuración del entorno y datos	29
4.1	Fuentes de datos	29
4.1.1	Mercado de Steam	29
4.1.2	BUFF y mercados externos	29
4.1.3	OCR e importación manual	30
4.2	Adquisición y persistencia	31
4.2.1	Pipeline de adquisición	31
4.2.2	Modelo de persistencia	31
4.3	Preparación de características	32
4.3.1	Variables de mercado	32
4.3.2	Dataset temporal de trading	32
4.3.3	Variables de cartera	33
4.4	Reglas económicas migradas	33
4.5	Simulador de cartera	34
4.5.1	Operaciones permitidas	34
4.5.2	Comisiones y valoración	34
5	Implementación y operación	35
5.1	Organización del sistema	35
5.2	Flujo operativo de adquisición	35
5.2.1	Sesiones, scraping y trazabilidad	36
5.2.2	Persistencia incremental	36
5.3	Consola de consulta	37
5.4	Reproducibilidad y configuración	37
6	Experimentación y análisis	39
6.1	Diseño experimental	39
6.1.1	Escenarios de evaluación	39
6.1.2	Estrategias de comparación	40
6.2	Validación técnica y pruebas	40
6.3	Métricas	41
6.3.1	Métricas de rentabilidad	41
6.3.2	Métricas de riesgo	41
6.4	Resultados	41
6.4.1	Migración de reglas económicas	41
6.4.2	Resultados del modelo supervisado	42
6.4.3	Evolución de los datasets de trading	42
6.4.4	Protocolo temporal reproducible	43
6.4.5	Ablación de histórico y aumentación	45
6.4.6	Comprobación del corte reciente	46
6.4.7	Evidencia de pruebas automatizadas	47

6.4.8	Validación funcional de OCR	47
6.4.9	Dataset de trading operativo	48
6.4.10	Dificultades y pivotaciones	49
6.4.11	Protocolo de episodios MARL	49
6.4.12	Resultados MARL	50
6.5	Experimentos finales pendientes	51
6.6	Discusión	51
6.6.1	Amenazas a la validez	51
6.6.2	Casos favorables	52
6.6.3	Limitaciones observadas	52
6.6.4	Lectura de los resultados	52
7	Conclusiones	53
7.1	Conclusiones principales	53
7.2	Cumplimiento de objetivos	53
7.3	Limitaciones	54
7.4	Trabajo futuro	54
	Bibliografía	55
A	Anexos	57
A.1	Estructura del repositorio	57
A.2	Configuración del entorno	57
A.3	Comandos de datasets y modelos	58
A.4	Parámetros de entrenamiento	58
A.5	Modelo de datos	59
A.6	Resultados adicionales	59

Índice de figuras

2.1	Pila tecnológica.	22
3.1	Capas del sistema.	25
6.1	Evolución de cortes.	43
6.2	Corte temporal de marzo.	44
6.3	Modelos en marzo.	45
6.4	Histórico y aumentación.	46
6.5	Métricas por umbral.	47
6.6	Trazas MARL.	51

Índice de tablas

3.1	Roles del sistema.	27
4.1	Controles del pipeline OCR.	31
4.2	Modelo de datos.	32
4.3	Reglas de simulación.	33
5.1	Módulos del sistema.	35
5.2	Vistas de consola.	37
6.1	Matriz experimental.	39
6.2	Cobertura de pruebas.	41
6.3	Umbrales de dirección.	42
6.4	Cortes de datos.	43
6.5	Configuraciones de marzo.	44
6.6	Comparativa de marzo.	45
6.7	Ablación logística.	46
6.8	Corte BUFF–Steam.	47
6.9	Pruebas automatizadas.	48
6.10	Validación funcional de OCR.	48
6.11	Episodio MARL.	49
6.12	Escenarios MARL.	50
A.1	Configuración experimental.	58

Lista de algoritmos

1	Ciclo de decisión	25
2	Decisión cooperativa	26
3	Extracción OCR.	30
4	Dataset temporal.	33
5	Flujo de scraping.	36
6	Ejecución de experimento.	37
7	Entrenamiento PPO	40

Siglas y abreviaturas

Esta relación recoge únicamente las siglas, códigos y abreviaturas que se emplean de forma recurrente.

API Interfaz que permite que dos programas intercambien datos o funciones.

BUFF Plataforma externa de compraventa de activos digitales, usada como fuente de precios en el caso de estudio.

CLAHE Técnica de mejora de contraste de una imagen por zonas, del inglés *Contrast Limited Adaptive Histogram Equalization*.

CLI Interfaz de línea de comandos, del inglés *Command-Line Interface*.

CNY Código internacional del yuan chino.

CS2 *Counter-Strike 2*, videojuego empleado como caso de estudio.

CSV Formato de datos tabulares separado por comas, del inglés *Comma-Separated Values*.

CTDE Entrenamiento centralizado y ejecución descentralizada, del inglés *Centralized Training with Decentralized Execution*.

EUR Código internacional del euro.

F1 Métrica que combina precisión y exhaustividad mediante su media armónica.

IA Inteligencia artificial.

MADDPG Algoritmo de aprendizaje por refuerzo multiagente, del inglés *Multi-Agent Deep Deterministic Policy Gradient*.

MAPPO Variante multiagente de PPO, del inglés *Multi-Agent Proximal Policy Optimization*.

MARL Aprendizaje por refuerzo multiagente, del inglés *Multi-Agent Reinforcement Learning*.

MDP Modelo matemático de decisión secuencial, del inglés *Markov Decision Process*.

OCR Reconocimiento de texto en imágenes, del inglés *Optical Character Recognition*.

PPO Algoritmo de optimización de políticas, del inglés *Proximal Policy Optimization*.

PR-AUC Área bajo la curva precisión-exhaustividad, del inglés *Precision-Recall Area Under the Curve*.

RL Aprendizaje por refuerzo, del inglés *Reinforcement Learning*.

RLlib Biblioteca de Ray utilizada para entrenar y evaluar políticas de aprendizaje por refuerzo.

ROC-AUC Área bajo la curva ROC, del inglés *Receiver Operating Characteristic Area Under the Curve*.

RSI Indicador de la intensidad de los cambios de precio, del inglés *Relative Strength Index*.

TFM Trabajo Fin de Máster.

Introducción

Los videojuegos con economías persistentes pueden generar mercados en los que sus bienes virtuales se intercambian dentro o fuera del propio juego. Este capítulo introduce el contexto de esos mercados, el problema de decisión que plantean y la propuesta del trabajo. Después concreta los objetivos, el alcance y la organización de la memoria.

1.1 Motivación

Un mercado de activos digitales en videojuegos es un entorno, integrado en el juego o accesible desde plataformas externas, donde las personas usuarias intercambian bienes virtuales. Esos bienes pueden ser objetos cosméticos, coleccionables, cajas u otros elementos vinculados a un juego. Su valor depende de la utilidad dentro del juego, de su apariencia, de su rareza, de la cantidad disponible y del interés de la comunidad. Por ello, aunque el bien sea digital, su precio puede variar con la oferta y la demanda. La literatura sobre bienes virtuales identifica precisamente dimensiones funcionales, estéticas y sociales como determinantes de su valor [1].

Estos mercados de activos digitales comparten varias características que dificultan una decisión de compra o venta (operación). Un mismo activo puede aparecer en varias plataformas, con precios, divisas, comisiones y disponibilidad diferentes. Además, conocer el precio no es suficiente para conocer el resultado de una operación: hay que considerar cuánto se pagaría al comprar, cuánto se recibiría al vender, cuánto tiempo quedaría inmovilizado el capital y si existen suficientes personas interesadas en intercambiar el activo.

En este contexto, una *oportunidad* es una posible operación que parece dejar un margen de beneficio después de considerar sus costes, si bien no es una garantía de beneficio. Una *señal* es un dato o una estimación que ayuda a valorar esa oportunidad, por ejemplo una diferencia de precio entre plataformas, el volumen de intercambios o una probabilidad calculada por un modelo.

En este trabajo propone un marco genérico para reunir esas señales, valorar sus costes y decidir si una operación debe revisarse, seguirse durante un tiempo o descartarse. El caso de estudio es el mercado de objetos de *Counter-Strike 2*, un videojuego con una economía persistente. Se ha elegido este videojuego porque ofrece activos con distintas calidades, plataformas con precios observables y restricciones prácticas que permiten evaluar el marco propuesto. Entre ellas se encuentran las comisiones, el cambio de moneda, la liquidez y el *trade hold*, un bloqueo temporal que impide intercambiar un artículo recién adquirido. Estas fricciones hacen que un margen aparente pueda desaparecer al calcular el ingreso neto, igual que en otros entornos de trading donde deben modelarse costes de transacción y restricciones de liquidez [2].

1.2 Planteamiento del problema

La comparación de precios entre plataformas plantea un problema único: decidir si un artículo merece estudiarse requiere reunir datos que llegan de fuentes distintas y aplicar sobre ellos las mismas reglas económicas. El punto de partida es un procedimiento manual en el que, para cada artículo, se anotaban el precio de compra, el de venta, la conversión de moneda, las comisiones, la fecha de desbloqueo y el capital bloqueado. Este procedimiento manual permite evaluar casos concretos, pero obligaba a repetir el proceso de cálculos y a comprobar datos por cada artículo.

Las diferentes plataformas muestran artículos puestos a la venta, su precio, el número de intercambios y sus características. Sin embargo, no siempre emplean la misma moneda, el mismo nombre o la misma fecha. Por ello, el problema no consiste solo en localizar una diferencia de precio. Hay que unificar la información, incorporar los costes y restricciones que afectan a la operación y responder con criterios consistentes a una pregunta sencilla: con los datos disponibles, ¿merece la pena estudiar este artículo?

El sistema propuesto convierte ese proceso en un flujo reproducible. Adquiere los datos, normaliza sus diferencias, aplica las reglas económicas, genera señales y simula el efecto de cada decisión sobre una cartera. De este modo, una posible operación se evalúa con los mismos criterios aunque proceda de fuentes distintas o se revise en momentos diferentes.

Cuando el sistema localiza un artículo en una o varias fuentes, lo incorpora como *candidato*. Tras recopilar sus precios y condiciones, puede clasificarlo en tres estados. *Revisar* indica que el margen son buenos y merece una comprobación manual. *Observar* indica que faltan datos o que conviene esperar una nueva captura (La captura se realiza periódicamente). *Bloquear* indica que los costes, el riesgo o las restricciones de cartera impiden continuar.

La dificultad principal está en justificar cada clasificación. No basta con indicar que un precio puede subir o bajar. La recomendación debe poder relacionarse con sus precios de entrada y salida, las comisiones, la liquidez, las reglas utilizadas y el estado de la cartera. Esta formulación conecta la estimación de señales con la gestión de cartera, donde retorno y riesgo deben analizarse conjuntamente [3].

Esta formulación organiza el proceso en tres fases. La separación permite evaluar cada parte por separado y conservar la trazabilidad desde una observación de mercado hasta la recomendación final:

1. **Observación y limpieza.** Reúne precios, anuncios, volumen y metadatos desde las fuentes disponibles. Después normaliza monedas, nombres y fechas, elimina duplicados y registra la calidad de cada captura. El resultado es un historial comparable, con la fuente y el instante de adquisición asociados a cada dato.
2. **Predicción.** Transforma ese historial en variables como el margen neto entre plataformas, la evolución del precio, la liquidez y la antigüedad de la captura. Con ellas, el modelo estima una probabilidad o puntuación que ayuda a priorizar cada artículo.

3. **Decisión.** Combina las señales con las reglas económicas y el estado de cartera dentro de un simulador. La arquitectura multiagente reparte la detección de oportunidades, la propuesta de acción y el control de riesgo. Su salida clasifica el artículo para revisión, observación o bloqueo.

1.3 Objetivos

El objetivo principal de este TFM es diseñar y evaluar una arquitectura multiagente para apoyar la toma de decisiones en mercados de activos digitales, usando como caso de estudio la economía virtual de *Counter-Strike 2*.

Los objetivos específicos se agrupan en cuatro bloques:

- **Comprensión del dominio:** analizar el mercado de activos digitales de *Counter-Strike 2*, sus plataformas, restricciones, comisiones, divisas, liquidez y efectos del *trade hold*.
- **Construcción de datos:** diseñar un flujo de adquisición y preparación de datos procedentes de Steam, BUFF, SteamDT, OCR y fuentes manuales, manteniendo trazabilidad de fuente, fecha, moneda y calidad del dato.
- **Simulación y predicción:** migrar reglas económicas del proceso manual a código comprobable, construir datasets temporales, entrenar modelos supervisados y generar señales probabilísticas que puedan usarse dentro del entorno de decisión.
- **Aprendizaje por refuerzo y decisión multiagente:** formular un entorno de aprendizaje por refuerzo con estado de cartera, acciones y recompensas definidos. Sobre esa base, distribuir los roles de detección, decisión y control de riesgo entre agentes especializados y evaluar su funcionamiento con métricas de rentabilidad, riesgo, exposición y capital inmovilizado.

1.4 Alcance y restricciones

El alcance del proyecto se centra en la simulación y en la evaluación de estrategias. Las decisiones generadas se registran con sus datos, reglas y resultado simulado. Este enfoque permite comparar propuestas de forma reproducible y estudiar su efecto sobre la cartera.

La arquitectura multiagente se plantea como una forma de separar responsabilidades dentro del proceso de decisión. La adquisición de datos, la predicción supervisada, el simulador económico, los agentes MARL y la evaluación experimental se mantienen como capas diferenciadas para que cada parte pueda validarse, sustituirse o compararse sin comprometer el conjunto del sistema.

Otra restricción relevante es la disponibilidad de histórico alineado entre plataformas. El mercado de Steam y el de BUFF no siempre ofrecen datos con la misma granularidad, moneda, horizonte o estabilidad de acceso. Por ello, una parte del trabajo consiste en construir una base experimental, donde los resultados preliminares se distingan claramente de una validación definitiva.

La metodología distingue entre el aprendizaje de patrones temporales y la evaluación operativa de oportunidades concretas. Esta separación permite aprovechar históricos más estables para entrenar modelos y, al mismo tiempo, incorporar precios puntuales de mercado cuando se simulan decisiones de cartera.

1.5 Metodología de desarrollo

El desarrollo se ha abordado de forma incremental. El primer paso fue convertir el procedimiento manual en reglas económicas explícitas. A continuación, se ha construido la base para adquirir, normalizar y conservar los datos de mercado. Con esos datos se han preparado conjuntos supervisados y se han evaluado modelos tabulares capaces de generar señales probabilísticas.

La última fase ha incorporado esas señales en un simulador de cartera y en un entorno multiagente compatible con PettingZoo y RLlib. Este orden permite comprobar primero los datos, las reglas y las métricas que intervienen en una decisión antes de estudiar la coordinación entre agentes mediante MARL. Así, cada capa aporta una evidencia verificable para la siguiente.

1.6 Estructura del documento

La memoria se estructura de forma secuencial siguiendo las fases principales del proyecto. A continuación, se detalla el contenido de cada capítulo:

- **Capítulo 1:** introduce el contexto del proyecto, motivación, planteamiento del problema, objetivos, alcance y metodología de desarrollo seguida.
- **Capítulo 2:** presenta el marco teórico necesario para entender el trabajo. Se tratan mercados de activos digitales, fricciones de mercado, RL, MARL, toma de decisiones en carteras, trabajos relacionados y tecnologías utilizadas.
- **Capítulo 3:** describe la arquitectura multiagente propuesta. Se presenta la formulación técnica del sistema, flujo general, entorno de decisión, agentes, función de recompensa y restricciones de riesgo.
- **Capítulo 4:** expone el entorno de datos y simulación. Se detallan fuentes de adquisición, persistencia, características utilizadas, simulador de cartera, OCR e interfaz de consulta.
- **Capítulo 5:** describe la implementación y operación del sistema. Presenta la organización del repositorio, el scraping, la persistencia, la consola local y la configuración reproducible.
- **Capítulo 6:** recoge la experimentación realizada. Incluye migración de reglas económicas, construcción de datasets supervisados, evaluación de modelos tabulares, pruebas técnicas y de rendimiento, dataset de trading, estado del entrenamiento MARL, dificultades y pivotaciones detectadas.

- **Capítulo 7:** presenta conclusiones principales, revisa el cumplimiento de objetivos y plantea líneas de trabajo necesarias para completar la validación experimental.
- **Anexos:** reúnen información complementaria sobre estructura del repositorio, configuración del entorno, comandos de ejecución, parámetros de entrenamiento y modelo de datos.

Marco teórico y estado del arte

Este capítulo reúne los conceptos necesarios para interpretar la propuesta: cómo funcionan los mercados de activos digitales, qué costes y restricciones afectan a una operación y cómo se emplean el aprendizaje por refuerzo y los agentes cooperativos en problemas de decisión. El caso de *Counter-Strike 2* se introduce como aplicación concreta del marco general.

2.1 Mercados de activos digitales

Un activo digital es un bien que existe dentro de un sistema informático y cuyo valor depende de reglas de acceso, escasez, utilidad percibida y capacidad de intercambio. En videojuegos, estos activos pueden tener una utilidad funcional, estética o social. Lehdonvirta estudia precisamente cómo los atributos funcionales, hedónicos y sociales influyen en la demanda de bienes virtuales [1]. Esta idea encaja bien con el mercado de *Counter-Strike 2*, donde dos objetos técnicamente similares pueden tener precios muy distintos por rareza, apariencia, estado, popularidad o valor simbólico dentro de la comunidad.

Las economías virtuales no son idénticas a los mercados financieros tradicionales, pero comparten algunos elementos: oferta limitada, negociación entre compradores y vendedores, formación de precios, costes de transacción y riesgo de liquidez. La diferencia principal es que el mercado está condicionado por una plataforma propietaria y por reglas del videojuego. El emisor del activo, las restricciones de intercambio y los cambios de diseño pueden alterar el valor de los objetos sin que exista una autoridad financiera externa que proteja al usuario.

2.1.1 Activos virtuales en Counter-Strike 2

En *Counter-Strike 2*, los activos más relevantes para este trabajo son skins, cuchillos, guantes, pegatinas, cajas y otros objetos cosméticos intercambiables. Su precio depende de factores como rareza, condición visual, popularidad del arma, volumen negociado, *float*, presencia de StatTrak, patrones especiales y demanda puntual. El *float* es el valor que representa la tasa de desgaste del aspecto. StatTrak identifica una versión que incorpora un contador de víctimas conseguidas con el arma. Estos atributos no siempre aparecen de forma limpia en una API, por lo que el sistema debe combinar scraping, histórico, normalización y revisión de nombres para representar correctamente cada variante.

El mercado no está concentrado en una única fuente. Steam actúa como mercado oficial y ofrece señales de precio e histórico, mientras que plataformas externas como BUFF permiten observar precios, *buy orders* y oportunidades que no aparecen con el mismo formato en Steam. Una *buy order* es una orden de compra publicada por un

usuario que indica el precio máximo que está dispuesto a pagar por un objeto. Esta fragmentación crea diferencias de precio entre plataformas, pero también aumenta la dificultad de decidir: una oportunidad puede existir en una fuente y desaparecer antes de que el activo pueda venderse en otra.

2.1.2 Fricciones de mercado

Las fricciones son costes o restricciones que separan el precio observado del beneficio neto. En este proyecto son especialmente importantes las comisiones de plataforma, la conversión entre EUR y CNY, el *spread*, el volumen disponible, las buy orders, el *trade hold* y el capital bloqueado. Por este motivo, el sistema no puede limitarse a comparar dos precios brutos. Debe calcular el precio neto de entrada, el precio neto de salida y el tiempo durante el cual la posición no puede cerrarse.

La liquidez es otra fricción central. Un activo puede mostrar un margen alto, pero si apenas tiene volumen o si las buy orders son débiles, la venta puede tardar demasiado o producirse a un precio inferior al esperado. En una cartera simulada, esa espera debe tener coste: el capital queda inmovilizado y no puede utilizarse para otras oportunidades.

2.2 Aprendizaje por refuerzo

El aprendizaje por refuerzo estudia problemas en los que un agente interactúa con un entorno, toma acciones y recibe recompensas. A diferencia del aprendizaje supervisado, donde el modelo aprende a reproducir etiquetas históricas, en RL el modelo aprende una política de actuación a partir de las consecuencias de sus acciones [4]. Esta formulación es útil cuando una decisión actual modifica el estado futuro, como ocurre al abrir una posición que queda bloqueada por *trade hold*.

2.2.1 Procesos de decisión de Markov

Un proceso de decisión de Markov, o MDP, se define mediante el estado, las acciones, la transición, la recompensa y el factor de descuento:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad 0 \leq \gamma < 1.$$

En esta expresión, $P(s' | s, a)$ representa la probabilidad de pasar del estado s al estado s' después de ejecutar la acción a , mientras que $R(s, a, s')$ representa la recompensa asociada a esa transición. El retorno acumulado desde el instante t se calcula por separado como:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

El estado resume la información disponible y la acción representa lo que el agente decide. En el contexto del TFM, el estado incluye variables de mercado y cartera, las acciones son comprar, vender, observar o mantener, y la recompensa se asocia al valor neto de la cartera simulada.

La hipótesis de Markov no implica que el sistema conozca todo el futuro, sino que el estado utilizado contiene la información necesaria para tomar la mejor decisión disponible en ese momento. Por eso es importante construir observaciones ricas: precio, spread, liquidez, probabilidad supervisada, capital disponible, posiciones abiertas y capital bloqueado.

2.2.2 Política, retorno y exploración

La política es la regla que transforma observaciones en acciones. El retorno mide la recompensa acumulada a lo largo del episodio. En un mercado con fricciones, maximizar la recompensa inmediata puede ser mala estrategia: una compra aparentemente rentable puede bloquear capital, aumentar exposición o no encontrar salida. Por ello, la recompensa debe mirar más allá del margen instantáneo y considerar el resultado de cartera.

La exploración permite probar acciones que todavía no se conocen bien. En un mercado de activos digitales con costes y restricciones, esta idea debe aplicarse con cuidado. En el TFM, la exploración se realiza en simulación, no con capital. Esta separación permite probar políticas y recompensas sin trasladar errores del entrenamiento al mercado.

2.3 Aprendizaje por refuerzo multiagente

El aprendizaje por refuerzo multiagente extiende la formulación anterior a varios agentes que interactúan dentro de un mismo entorno. Esta ampliación introduce dificultades nuevas: cada agente aprende mientras los demás también cambian, por lo que el entorno puede volverse no estacionario desde el punto de vista individual. Lowe et al. señalan este problema y proponen entrenar agentes con información adicional durante el entrenamiento para mejorar la coordinación mediante MADDPG [5].

La decisión de cartera mezcla tareas de naturaleza distinta: detectar oportunidades, ejecutar acciones concretas y controlar exposición global. Separar estas responsabilidades permite que cada agente trabaje con un punto de vista más acotado, siempre que la recompensa cooperativa mantenga alineado el objetivo común.

2.3.1 Cooperación y coordinación entre agentes

La cooperación aparece cuando los agentes comparten un objetivo global. Scout puede detectar oportunidades prometedoras, Trader puede decidir la acción operativa y Portfolio puede limitar decisiones que deterioren la cartera. Ninguno de estos roles tiene sentido de forma aislada si el resultado final no mejora el valor neto ajustado por riesgo.

La coordinación es necesaria porque una decisión localmente buena puede ser globalmente mala. Por ejemplo, comprar un activo con spread atractivo puede ser razonable para Trader, pero no para Portfolio si ya existe demasiada exposición a ese tipo de objeto o si el capital bloqueado supera un límite aceptable. Esta tensión puede abordarse con recompensas específicas por rol o con una recompensa cooperativa vinculada a la cartera.

Para N agentes, cada paso combina observaciones y acciones locales. La implementación actual utiliza una recompensa cooperativa común, calculada a partir del resultado de cartera y asignada por igual a todos los agentes:

$$\mathbf{o}_t = (o_t^1, \dots, o_t^N), \quad \mathbf{a}_t = (a_t^1, \dots, a_t^N), \quad r_t^i = R_{\text{cartera},t} \quad \forall i \in \{1, \dots, N\}.$$

En esta expresión, N es el número de agentes, que en el prototipo es tres. El vector $\mathbf{o}_t$ reúne las observaciones de Scout, Trader y Portfolio en el instante t , mientras que $\mathbf{a}_t$ reúne las acciones que han propuesto. El valor r_t^i es la recompensa que recibe el agente i . Como todos los valores r_t^i se igualan a $R_{\text{cartera},t}$, los tres agentes reciben el mismo resultado en cada paso.

$R_{\text{cartera},t}$ es un valor único calculado con el beneficio realizado, el retorno de una operación ejecutada y penalizaciones por inactividad, límites de riesgo incumplidos, *drawdown*, capital bloqueado y volatilidad. Un resultado favorable aumenta la recompensa, mientras que una compra que eleva el riesgo o inmoviliza demasiado capital la reduce. Esta elección mantiene a los agentes alineados con el mismo objetivo de cartera. Cada política recibe solo su observación o_t^i , mientras que el estado global se conserva para diagnóstico y para ampliar el entrenamiento con un crítico centralizado cuando sea necesario.

2.3.2 Entrenamiento centralizado y ejecución descentralizada

El paradigma de entrenamiento centralizado y ejecución descentralizada, conocido como CTDE, permite entrenar con más información de la que cada agente tendrá en ejecución. La idea es que durante el entrenamiento puede existir un crítico o una función de valor con acceso al estado global, mientras que en ejecución cada actor decide con su observación local. Este enfoque aparece en trabajos multiagente como MADDPG [5] y en variantes cooperativas basadas en PPO como MAPPO [6].

La elección de PPO como base práctica se debe a su estabilidad y a su disponibilidad en herramientas maduras. Schulman et al. introducen PPO como un método de gradiente de política que limita actualizaciones demasiado agresivas [7]. Para una acción observada, el cociente entre la política nueva y la anterior es:

$$r_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)}, \quad L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right].$$

El recorte evita que una actualización cambie en exceso la probabilidad de una acción. Posteriormente, Yu et al. muestran que PPO puede funcionar como baseline fuerte en juegos cooperativos multiagente cuando se implementa con cuidado [6]. En el proyecto, esta línea se materializa en un entorno PettingZoo y un wrapper para RLlib, que permiten integrar entrenamientos con PPO y MAPPO.

2.4 Toma de decisiones en carteras

La gestión de una cartera no consiste solo en elegir activos con rentabilidad esperada positiva. También debe controlar riesgo, concentración, liquidez, costes de transacción y horizonte temporal. La teoría moderna de carteras introdujo la idea de estudiar rentabilidad y riesgo conjuntamente, en lugar de analizar cada inversión de forma aislada [3]. Aunque el mercado de CS2 no sea un mercado bursátil regulado, la intuición sigue siendo útil: una cartera puede empeorar aunque una operación individual parezca atractiva.

2.4.1 Rentabilidad y riesgo

En este TFM, la rentabilidad se mide mediante beneficio neto, retorno porcentual, saldo final y evolución del valor de cartera. El riesgo se observa mediante drawdown, capital bloqueado, exposición por activo o plataforma y operaciones rechazadas por restricciones. Estas métricas son más informativas que una simple precisión de predicción, porque conectan el modelo con el resultado económico simulado.

El aprendizaje supervisado puede estimar una probabilidad útil, pero esa probabilidad no es una decisión. Una señal de subida o de spread rentable debe pasar por el simulador, por las comisiones, por la liquidez y por el estado de cartera. Esta separación evita que el sistema confunda predecir con invertir.

2.4.2 Liquidez y costes de transacción

Los costes de transacción son centrales en cualquier sistema de trading. En un mercado de baja o media liquidez, pequeñas diferencias de precio pueden desaparecer al aplicar comisiones, conversión de moneda o retrasos de venta. FinRL incorpora explícitamente restricciones como costes de transacción, liquidez y aversión al riesgo en entornos de trading con aprendizaje por refuerzo [2]. Esa orientación resulta útil para este TFM, aunque el dominio sea distinto: el objetivo no es copiar un framework financiero, sino adoptar la disciplina de simular fricciones del mercado.

En el caso CS2, las fricciones se vuelven todavía más visibles por el *trade hold*. Durante varios días, una posición comprada no puede venderse, aunque el precio cambie. Esta restricción convierte una oportunidad puntual en una decisión temporal: el sistema debe estimar no solo el margen de entrada, sino la probabilidad de que el margen siga siendo válido cuando la posición pueda cerrarse.

2.5 Aplicaciones relacionadas

Los trabajos de RL financiero muestran que entrenar agentes de trading exige reproducibilidad, backtesting y control de fricciones [2]. Los trabajos de MARL muestran que la coordinación entre agentes puede ser útil, pero también introduce inestabilidad y necesidad de evaluación cuidadosa [5, 6]. El proyecto se sitúa en la intersección de ambas ideas: usa un mercado de activos digitales como entorno de decisión, incorpora señales supervisadas y plantea una arquitectura multiagente cooperativa.

La diferencia principal respecto a una aplicación financiera clásica es el dominio. Los activos de CS2 son bienes virtuales controlados por plataformas, con liquidez fragmen-

tada y reglas propias. La diferencia respecto a una simple herramienta de scraping es el objetivo: no basta con mostrar precios, sino que se busca evaluar decisiones de cartera bajo restricciones. Por ello, el trabajo combina adquisición de datos, simulación económica, predicción supervisada y aprendizaje por refuerzo multiagente dentro de una misma metodología.

2.6 Tecnologías utilizadas

Las tecnologías se han seleccionado por su función dentro del sistema. Cada una cubre una necesidad concreta de adquisición, persistencia, predicción, simulación, entrenamiento o validación. El objetivo es mantener un flujo reproducible en el que los datos, los modelos y las decisiones simuladas puedan revisarse de forma independiente.

Bloque	Tecnologías principales
Adquisición	Python, Playwright, HTTPX, Steam, SteamDT, BUFF y OCR.
Persistencia	Postgres/Supabase, SQLAlchemy, AsyncPG y PyArrow.
Predicción	Pandas, NumPy, scikit-learn y Joblib.
Simulación	Módulos propios de cartera, riesgo, comisiones y <i>trade hold</i> .
Multiagente	PettingZoo, Ray/RLlib, Gymnasium, PyTorch y PPO.
Validación	Pytest, pytest-asyncio, Ruff, Mypy y pruebas de integración.
Interfaz	HTML, CSS, JavaScript y servidor web local en Python.

Figura 2.1: Pila tecnológica.

2.6.1 Herramientas de desarrollo

El proyecto se desarrolla como un monorepo en Python 3.11. Git se utiliza para versionar cambios y mantener trazabilidad de las decisiones técnicas. La estructura separa aplicaciones ejecutables, paquetes reutilizables, documentación, datos derivados y pruebas, lo que permite evolucionar el sistema sin mezclar adquisición, predicción, simulación y evaluación.

Pytest y pytest-asyncio se emplean para automatizar pruebas unitarias y asíncronas. Ruff se usa como herramienta de análisis estático y estilo, mientras que Mypy permite detectar errores de tipado en módulos críticos. Estas herramientas no forman parte de la lógica de negocio, pero sí de la evidencia de calidad del desarrollo.

2.6.2 Adquisición y tratamiento de datos

La adquisición de datos se implementa principalmente en Python. HTTPX permite consultar fuentes HTTP cuando existe una interfaz estable, mientras que Playwright se reserva para escenarios en los que el dato solo está disponible mediante navegación o renderizado de página. En el caso del OCR, OpenCV y Tesseract permiten extraer información visual cuando el dato no se recibe en formato estructurado.

Pandas, NumPy y PyArrow se utilizan para preparar conjuntos de datos, generar características temporales y conservar datasets derivados en formatos reproducibles. Esta capa conecta las fuentes de mercado con los modelos supervisados y con los episodios del simulador.

2.6.3 Persistencia y modelo de datos

Postgres, a través de Supabase en el planteamiento operativo, actúa como base de persistencia para artículos, observaciones de mercado, históricos, predicciones y decisiones simuladas. AsyncPG y SQLAlchemy facilitan el acceso desde Python y permiten separar consultas, repositorios y lógica de dominio.

La elección de una base relacional se ajusta al tipo de información del proyecto: series temporales, activos, plataformas, precios, señales y decisiones deben poder relacionarse y auditarse posteriormente. Esta trazabilidad es importante para explicar por qué una oportunidad ha sido recomendada, observada o descartada.

2.6.4 Predicción, simulación y entrenamiento multiagente

scikit-learn se utiliza para los modelos supervisados tabulares, especialmente regresión logística, random forest e histogram gradient boosting. Joblib permite persistir modelos entrenados para usarlos después en inferencia. Estas tecnologías cubren la parte predictiva, pero la decisión final se evalúa dentro del simulador de cartera.

La simulación económica se implementa con módulos propios porque las reglas del dominio son específicas: comisiones, conversión de moneda, beneficio neto, *trade hold*, capital bloqueado y restricciones de exposición. Sobre esta base se construye el entorno multiagente con PettingZoo y el adaptador de entrenamiento con Ray/RLlib. PyTorch y Gymnasium forman parte del ecosistema de entrenamiento usado por RLlib.

2.6.5 Interfaz y operación local

La interfaz web local se implementa con HTML, CSS y JavaScript, servida desde Python. Centraliza el estado de scraping, los datos disponibles, las recomendaciones, las métricas y las tareas operativas, facilitando la observabilidad del sistema durante el desarrollo y la evaluación.

Arquitectura multiagente propuesta

En este capítulo se describe la arquitectura planteada para convertir datos de mercado en decisiones simuladas. Se presentan sus capas, el flujo de información, el entorno de cartera y la distribución de responsabilidades entre los agentes Scout, Trader y Portfolio.

3.1 Formulación técnica del sistema

A partir del planteamiento descrito en el Capítulo 1, la propuesta traduce el caso de uso a una tarea de decisión secuencial sobre una cartera simulada de activos digitales, siguiendo la formulación general de los problemas de aprendizaje por refuerzo [4]. En cada instante, el sistema recibe observaciones de mercado, estado de cartera y señales derivadas. A partir de esa información decide si conviene comprar, vender, mantener en observación o rechazar una oportunidad.

La formulación técnica incorpora las fricciones que condicionan cada acción: beneficio neto después de comisiones, cambio de moneda, liquidez disponible, capital ya bloqueado, tiempo de desbloqueo por *trade hold* y exposición acumulada a un mismo activo o plataforma. Con estos elementos, el objetivo del sistema es maximizar el valor total de la cartera simulada bajo restricciones de riesgo y fricción de mercado.

La arquitectura propuesta separa esta formulación en dos niveles. El primer nivel observa y transforma el mercado: extrae datos, los normaliza, los guarda y calcula señales supervisadas. El segundo nivel toma decisiones dentro de un simulador: recibe observaciones ya preparadas, emite acciones y recibe recompensas según el comportamiento de la cartera. Esta separación permite aislar la adquisición de datos, la limpieza, la predicción y el control de riesgo como responsabilidades diferenciadas.

3.2 Visión general del sistema

La arquitectura se organiza en cinco capas: adquisición, persistencia, predicción, decisión multiagente y simulación. Cada capa tiene una responsabilidad concreta y se comunica con la siguiente mediante datos normalizados. Esta decisión de diseño reduce el acoplamiento y permite validar cada parte por separado, una práctica habitual en entornos de trading reproducibles y backtesting [2].

La decisión final siempre queda registrada como simulada. Esto permite evaluar el sistema con métricas de cartera sin exponer capital durante la investigación. La arquitectura puede generar recomendaciones, episodios y métricas, pero no envía órdenes a Steam, BUFF ni ninguna otra plataforma.

1. Adquisición	Steam, BUFF, SteamDT, OCR e importaciones manuales capturan observaciones.
2. Persistencia	Postgres/Supabase guarda artículos, históricos, snapshots, predicciones y decisiones simuladas.
3. Predicción	El modelo supervisado genera probabilidades calibradas y señales de mercado.
4. Decisión MARL	Scout, Trader y Portfolio deciden dentro del entorno PettingZoo/RLlib.
5. Simulación	La cartera aplica comisiones, divisas, <i>trade hold</i> , riesgo y métricas.

Figura 3.1: Capas del sistema.

3.3 Flujo operativo

El flujo operativo comienza con artículos localizados mediante una búsqueda de mercado o una fuente de descubrimiento. Los agentes extractores consultan las fuentes disponibles, capturan precios, buy orders, volumen y metadatos, y construyen observaciones normalizadas. Después, la persistencia conserva el histórico en formato auditable para que el dataset pueda reconstruirse sin depender de una sesión concreta de scraping.

Sobre ese histórico se construyen características de mercado. El modelo supervisado produce una probabilidad calibrada de oportunidad rentable en un horizonte determinado. Esa probabilidad no se interpreta como una orden de compra, sino como una característica adicional que los agentes MARL pueden utilizar. Finalmente, el entorno multiagente genera una observación para cada política, recoge sus acciones y aplica las reglas del simulador.

Algoritmo 1 Ciclo de decisión

- 1: Recoger observaciones recientes del mercado.
 - 2: Normalizar precios, moneda, liquidez y metadatos de plataforma.
 - 3: Actualizar el estado de cartera y las variables de riesgo.
 - 4: Calcular señales supervisadas disponibles para el episodio.
 - 5: Generar una observación local para Scout, Trader y Portfolio.
 - 6: Obtener acciones de los agentes y aplicar restricciones de riesgo.
 - 7: Simular la decisión final sobre la cartera.
 - 8: Calcular recompensa, métricas y trazabilidad del episodio.
-

3.4 Definición del entorno

El entorno representa una cartera simulada que evoluciona a partir de datos históricos. Cada episodio contiene una secuencia de observaciones de mercado y un estado interno formado por saldo disponible, posiciones abiertas, posiciones bloqueadas, fechas de desbloqueo y métricas de exposición.

3.4.1 Estado y observaciones

Las observaciones combinan variables de mercado y variables de cartera. Entre las variables de mercado se incluyen precios de Steam y BUFF normalizados a EUR, precios originales en su moneda nativa, volumen, liquidez, spread bruto, spread neto, buy orders, edad de la variante y señales temporales como retornos, medias móviles, RSI o desviaciones de ventas. Entre las variables de cartera se incluyen efectivo disponible, capital bloqueado, posiciones abiertas, exposición por activo y estado de desbloqueo.

La salida del modelo supervisado se incorpora como una probabilidad calibrada de que una oportunidad mantenga un spread rentable en el horizonte definido. Esta señal permite aprovechar aprendizaje supervisado sin convertirlo en una decisión rígida. Si la probabilidad es alta pero la cartera está sobreexpuesta o el capital está bloqueado, el entorno puede rechazar o posponer la operación.

3.4.2 Espacio de acciones

El espacio de acciones se mantiene deliberadamente simple. Las acciones principales son comprar, vender, mantener y observar. En la parte de compra o venta puede añadirse un tamaño de posición discreto para limitar el número de combinaciones. Esta simplificación permite entrenar y depurar el entorno antes de introducir acciones continuas o reglas de asignación más complejas.

En la versión implementada, Scout puede ignorar o marcar una oportunidad, Trader puede mantener o solicitar la compra de una unidad y Portfolio puede rechazar o aprobar el riesgo. La compra solo se ejecuta cuando los tres agentes eligen la acción favorable y el candidato cumple los límites.

Algoritmo 2 Decisión cooperativa

Entrada: Candidato c_t , estado de cartera s_t y observaciones locales

Salida: Transición de cartera y recompensa compartida R_t

```
1:  $a_t^S \leftarrow \pi_S(o_t^S)$ 
2:  $a_t^T \leftarrow \pi_T(o_t^T, m_t^T)$ 
3:  $a_t^P \leftarrow \pi_P(o_t^P, m_t^P)$ 
4: if  $a_t^S = \text{marcar}$  and  $a_t^T = \text{comprar}$  and  $a_t^P = \text{aprobar}$  and  $c_t$  cumple los límites
   then
5:   Abrir una posición simulada y bloquear su capital
6: else
7:   Mantener el saldo y registrar el motivo de no ejecución
8: end if
9: Actualizar posiciones liquidables, saldo, exposición y drawdown
10: Calcular  $R_t$  y devolver la misma recompensa a los tres agentes
```

Las tres primeras líneas recogen la acción propuesta por cada política: Scout marca o ignora, Trader mantiene o solicita la compra y Portfolio aprueba o rechaza el riesgo. Las variables m_t^T y m_t^P incorporan las métricas de mercado y cartera que necesita cada rol para decidir.

3.4.3 Transición del entorno

Cuando se ejecuta una compra simulada, el saldo disponible disminuye y se abre una posición. Esa posición queda bloqueada durante el periodo de *trade hold*. Mientras está bloqueada no puede venderse, pero sí afecta a la exposición y al capital inmovilizado. Cuando se ejecuta una venta simulada, el simulador calcula el ingreso neto aplicando comisiones, conversión de moneda y reglas de valoración. El estado de cartera se actualiza después de cada paso.

3.5 Agentes del sistema

La arquitectura distingue entre agentes extractores y agentes MARL. Los extractores no se entrenan mediante aprendizaje por refuerzo y no deciden operaciones. Su trabajo es adquirir datos desde Steam, BUFF, SteamDT, OCR o CSV/API y transformarlos en observaciones persistidas. Los agentes MARL, en cambio, operan dentro del entorno de simulación.

Agente	Función
<i>Adquisición de datos</i>	
Steam y BUFF	Recogen precios, histórico, buy orders y volumen.
SteamDT	Descubre candidatos para priorizar la captura.
OCR y CSV	Convierten capturas y archivos en observaciones.
<i>Decisión multiagente</i>	
Scout	Filtra oportunidades por margen, liquidez y señal.
Trader	Propone comprar, vender, mantener u observar.
Portfolio	Valida exposición, capital bloqueado y riesgo.

Tabla 3.1: Roles del sistema.

3.5.1 Scout

Scout se encarga de detectar oportunidades. Su observación está orientada a señales de precio, spread, liquidez y probabilidad supervisada. Su papel no es decidir toda la operación, sino marcar qué activos parecen merecer atención dentro del episodio.

3.5.2 Trader

Trader decide la acción operativa principal: comprar, vender, mantener u observar. Para ello considera la señal de Scout, el estado de mercado, el posible beneficio neto y el tamaño de posición permitido. Su responsabilidad está más cerca de la ejecución de una oportunidad concreta.

3.5.3 Portfolio

Portfolio controla la coherencia global de la cartera. Su observación incorpora saldo, capital bloqueado, posiciones abiertas, exposición máxima y restricciones de riesgo. Este

agente puede penalizar o bloquear decisiones que parezcan rentables de forma aislada pero que deterioren el perfil global de la cartera.

3.6 Función de recompensa

La recompensa debe reflejar el objetivo del sistema: mejorar el valor neto de la cartera sin ignorar riesgo ni fricción. Una recompensa basada solo en comprar barato o en detectar subida de precio sería incompleta, porque no tendría en cuenta si la operación puede cerrarse, si el capital queda bloqueado o si el spread desaparece antes de poder vender.

3.6.1 Recompensa individual

Las señales individuales pueden ayudar a entrenar comportamientos especializados. Scout puede recibir recompensa por identificar oportunidades que más adelante resultan útiles. Trader puede recibirla por ejecutar acciones con beneficio neto y Portfolio por evitar sobreexposición, operaciones ilíquidas o capital bloqueado excesivo. Estas recompensas no sustituyen al objetivo global, pero pueden guiar el aprendizaje de cada rol.

3.6.2 Recompensa cooperativa

La recompensa cooperativa se vincula al valor total de la cartera simulada. Debe considerar beneficio realizado, valoración de posiciones abiertas, capital bloqueado, drawdown, operaciones rechazadas por riesgo y costes de transacción, en línea con el tratamiento conjunto de rentabilidad y riesgo de la teoría de carteras [3]. En la implementación actual, el entorno MARL ya expone métricas como recompensa media, operaciones, efectivo, beneficio realizado, drawdown, capital bloqueado y posiciones abiertas.

3.7 Restricciones de riesgo

Las restricciones de riesgo se introducen antes de escalar el entrenamiento. Las más importantes son: no usar capital inexistente, limitar exposición por activo o plataforma, rechazar operaciones con liquidez insuficiente, respetar el *trade hold*, controlar el capital bloqueado y separar claramente simulación de operación. Estas reglas son sencillas, pero evitan que los agentes aprendan políticas imposibles de ejecutar en el mercado.

La salida del sistema debe ser siempre auditable. Cada predicción, acción o decisión simulada debe poder relacionarse con su fuente de datos, versión de modelo, episodio y configuración de simulación. Esta trazabilidad es necesaria para interpretar resultados y para distinguir entre error de datos, error de modelo y mala política de decisión.

Configuración del entorno y datos

Este capítulo explica qué datos utiliza el sistema, cómo se obtienen y cómo se preparan antes de emplearlos en la simulación o en los modelos. También presenta las reglas económicas y las variables que describen el estado de una cartera.

4.1 Fuentes de datos

El sistema trabaja con varias fuentes porque el mercado de CS2 está fragmentado. Ninguna fuente aislada contiene toda la información necesaria para decidir. Steam aporta precio, histórico y señales del mercado principal. BUFF permite comparar oportunidades y buy orders. SteamDT ayuda a descubrir candidatos. El OCR permite capturar datos cuando no existe una API cómoda. Los CSV o importaciones manuales sirven para integrar muestras controladas o material histórico.

4.1.1 Mercado de Steam

Steam actúa como una de las fuentes principales de precio e histórico. En el proyecto se capturan variantes de artículos, precios actuales, moneda, buy orders y puntos históricos cuando están disponibles. La información se normaliza para que pueda combinarse con otras plataformas y para evitar que las decisiones dependan de nombres o formatos propios de una fuente.

En la práctica, Steam no se usa solo como una tabla de precios. También aporta señales de liquidez y de comportamiento temporal. El histórico permite construir retornos, medias móviles, volatilidad y desviaciones frente a ventas recientes. Estas variables son necesarias para que el modelo no mire únicamente una fotografía puntual del mercado.

4.1.2 BUFF y mercados externos

BUFF se utiliza como mercado externo de comparación. Sus precios se reciben principalmente en CNY, por lo que el sistema conserva tanto el valor original como su conversión a EUR. Esta doble representación permite auditar el dato original y, al mismo tiempo, entrenar modelos sobre una moneda canónica. Las buy orders de BUFF son especialmente útiles porque ayudan a estimar si una salida tiene demanda o si el precio observado es poco ejecutable.

SteamDT se utiliza como apoyo para descubrir candidatos y estrategias de búsqueda, no como fuente única de decisión. Su función dentro del sistema es reducir el espacio de búsqueda inicial y ayudar a priorizar qué artículos merece la pena observar con más profundidad.

Tras las primeras pruebas, BUFF se separa del entrenamiento principal. El histórico masivo se apoya en Steam, mientras que BUFF se usa como precio vivo de entrada

cuando se evalúa un flip concreto hacia Steam. De este modo el modelo no intenta predecir el precio de BUFF ni depende de scraping recurrente de BUFF para aprender patrones temporales.

4.1.3 OCR e importación manual

El OCR se incorpora como vía secundaria de adquisición. Su objetivo es convertir capturas, tablas o interfaces no estructuradas en observaciones equivalentes a las obtenidas por scraping o API. El flujo incluye preprocesado de imagen, reconocimiento de texto, parsing estructurado, validación y persistencia. Aunque es más frágil que una fuente estructurada, resulta útil cuando una plataforma no expone datos cómodamente o cuando se quiere incorporar material histórico procedente de capturas.

La implementación actual se centra en tablas y mensajes de mercado, que son las capturas con una estructura suficientemente regular para poder validarse. OpenCV amplía la imagen por un factor dos, la convierte a escala de grises, mejora el contraste local con CLAHE, reduce ruido y aplica una binarización adaptativa. CLAHE divide la imagen en zonas, ajusta el contraste de cada una y limita ese refuerzo para no amplificar el ruido. Tesseract devuelve las palabras y una confianza por palabra. La confianza agregada utilizada por el pipeline es la media de las n palabras reconocidas con confianza válida:

$$c_{\text{OCR}} = \frac{1}{100n} \sum_{j=1}^n c_j.$$

El parser normaliza espacios y separadores decimales, identifica precio, moneda, calidad, plataforma y volumen cuando aparecen en la línea. Solo se convierten en observaciones las extracciones con $c_{\text{OCR}} \geq 0,5$ y con un precio positivo menor de un millón de unidades monetarias. Esta regla evita que una captura dudosa entre silenciosamente al histórico.

Algoritmo 3 Extracción OCR.

Entrada: Captura I , confianza mínima τ y contexto de fuente

Salida: Observaciones normalizadas o rechazo trazable

```

1: Cargar  $I$  y aplicar escala, contraste, reducción de ruido y binarización
2: Ejecutar Tesseract y calcular  $c_{\text{OCR}}$ 
3: if  $c_{\text{OCR}} < \tau$  then
4:     Rechazar la captura y conservar su referencia para revisión
5: else
6:     for all líneas reconocidas do
7:         Extraer artículo, calidad, plataforma, precio, moneda y volumen
8:         if precio y moneda son válidos then
9:             Construir identificador canónico y observación con la confianza obtenida
10:        end if
11:    end for
12: end if

```

Etapa	Salida	Control aplicado
Preprocesado	Imagen binaria ampliada	CLAHE, reducción de ruido y umbral adaptativo.
Reconocimiento	Texto y confianza por palabra	Media de confianzas válidas de Tesseract.
Parsing	Campos de mercado normalizados	Precio, moneda, calidad, plataforma y volumen.
Validación	Observación o rechazo	Confianza mínima, precio positivo y rango máximo.

Tabla 4.1: Controles del pipeline OCR.

Las importaciones manuales y CSV cumplen una función complementaria. Permiten introducir muestras controladas, fixtures o históricos parciales sin depender de una sesión de navegador. Esta vía ha sido útil para validar parsers, modelos de persistencia y reglas económicas antes de automatizar el flujo completo.

4.2 Adquisición y persistencia

Esta sección describe el recorrido de un dato desde su captura hasta su almacenamiento. El objetivo es conservar observaciones comparables y trazables, de modo que puedan utilizarse más adelante en la preparación de variables y en los experimentos.

4.2.1 Pipeline de adquisición

La adquisición se organiza como una capa de agentes extractores. Estos procesos pueden ejecutarse como comandos CLI, jobs o servicios programados. Su función es capturar datos, normalizarlos, validar errores y guardarlos con trazabilidad. El flujo incluye scraping, refresco de históricos, OCR, importación manual y un bucle operativo local que puede ejecutar scraping y actualización periódica.

La adquisición periódica contempla cookies o sesiones, delays configurables, límites de concurrencia, reintentos, deduplicación y registro de anomalías. Esta parte es importante porque las fuentes no siempre son estables: pueden cambiar el formato, limitar el ritmo de acceso o devolver información incompleta.

4.2.2 Modelo de persistencia

La persistencia se apoya en Supabase/Postgres. La decisión de diseño es tratar la base de datos como fuente de verdad y mantener el histórico de forma append-only siempre que sea posible. Las tablas operativas principales son `market_items` y `market_history_points`. La primera mantiene una fila por variante de artículo y estado actual por plataforma. La segunda guarda series temporales en formato largo por artículo, plataforma, fecha y métrica.

También se define un esquema canónico más amplio con entidades como `assets`, `platforms`, `market_observations`, `predictions`, `investment_decisions` y

`simulated_positions`. Esta separación permite evolucionar desde el esquema operativo actual hacia una representación más trazable y auditable.

Entidad	Uso dentro del TFM
<code>market_items</code>	Estado actual de cada variante por plataforma.
<code>market_history_points</code>	Serie temporal en formato largo para precios, buy orders y métricas.
<code>predictions</code>	Salidas versionadas del modelo supervisado.
<code>investment_decisions</code>	Decisiones simuladas y razones asociadas.
<code>simulated_positions</code>	Posiciones abiertas, bloqueadas o cerradas por el simulador.

Tabla 4.2: Modelo de datos.

4.3 Preparación de características

Antes de entrenar un modelo o simular una cartera, las observaciones se transforman en variables que resumen precios, liquidez, evolución temporal y estado de la cartera. A continuación se explican las variables empleadas y la construcción temporal de las etiquetas.

4.3.1 Variables de mercado

Las características de mercado incluyen precios Steam y BUFF normalizados, precios originales, spread bruto, spread neto, buy orders, volumen, liquidez, número de listings, antigüedad de variante y señales temporales. En la fase supervisada se han explorado variables de momentum y reversión como retornos a 1, 3 y 7 días, distancia a medias móviles, RSI, volatilidad y desviaciones de ventas. Esta combinación de señales temporales y restricciones operativas sigue la misma lógica que los entornos de trading con control de fricciones [2].

La exploración inicial detectó que algunas señales temporales aportan información moderada, mientras que ciertas variables categóricas de alta cardinalidad, como identificadores concretos de skins, deben tratarse con cuidado para evitar sobreajuste. Por ello, el entrenamiento separa variables de evaluación y variables de entrenamiento, y permite realizar ablations sin reconstruir todo el dataset.

4.3.2 Dataset temporal de trading

Para cada observación de un artículo en el día t , el constructor busca la primera salida disponible a partir de $t + h$, donde $h = 8$ días representa el bloqueo aplicado por el simulador. La etiqueta no representa una subida de precio aislada, sino una operación que supera los umbrales económicos tras aplicar comisiones y conversión:

$$y_t = 1 \iff \pi_{t+h} \geq \pi_{\text{mín}} \wedge \rho_{t+h} \geq \rho_{\text{mín}}.$$

Los conjuntos se separan por fecha y se elimina una franja de purga antes de cada frontera para que una salida futura no coincida con el entrenamiento de un periodo anterior.

Algoritmo 4 Dataset temporal.

Entrada: Históricos Steam/BUFF, horizonte h , fechas de corte y purga g

Salida: Conjuntos de entrenamiento, validación y prueba trazables

- 1: **for all** observaciones (i, t) ordenadas por artículo y fecha **do**
 - 2: Calcular variables disponibles hasta t
 - 3: Buscar la primera salida del artículo i a partir de $t + h$
 - 4: **if** la salida es válida **then**
 - 5: Calcular π_{t+h} , ρ_{t+h} y la etiqueta y_t
 - 6: Conservar (i, t) , sus variables y la fecha de salida
 - 7: **end if**
 - 8: **end for**
 - 9: Excluir g días antes de cada frontera temporal
 - 10: Separar por fecha y guardar filas, artículos y rango de cada conjunto
-

4.3.3 Variables de cartera

Las variables de cartera proceden del simulador. Incluyen saldo disponible, valor de posiciones abiertas, capital bloqueado por *trade hold*, fecha de desbloqueo, exposición por activo, exposición por plataforma, beneficio realizado y drawdown. Estas variables son necesarias para que el agente Portfolio no decida únicamente por oportunidad local, sino por estado global de la cartera, coherentemente con la evaluación conjunta de rentabilidad y riesgo [3].

4.4 Reglas económicas migradas

El simulador formaliza las reglas económicas identificadas en el procedimiento manual de seguimiento. Incluye conversión EUR/CNY, comisiones, beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado.

Regla	Uso en la simulación
Conversión EUR/CNY	Normaliza precios de Steam y BUFF para compararlos en una moneda común.
Fee BUFF 0,975	Estima el ingreso neto al vender en BUFF.
Fee Steam 0,87	Representa la comisión de mercado en ventas Steam.
Cash-out conservador	Permite simular pérdida adicional al convertir saldo Steam en liquidez externa.
<i>Trade hold</i>	Bloquea posiciones durante un horizonte conservador de ocho días.
Beneficio neto	Calcula diferencia entre ingreso neto de venta y coste de entrada.

Tabla 4.3: Reglas de simulación.

4.5 Simulador de cartera

El simulador representa las consecuencias de una decisión sobre el efectivo disponible, las posiciones abiertas y el riesgo. Sus reglas permiten comparar alternativas con las mismas comisiones, bloqueos y límites de exposición.

4.5.1 Operaciones permitidas

Las operaciones básicas son comprar, vender y mantener. Una compra abre una posición con precio de entrada, cantidad, plataforma y fecha de bloqueo. Una venta cierra una posición si está disponible y calcula el beneficio realizado. Mantener no modifica la posición, pero puede afectar al rendimiento si el mercado se mueve o si el capital sigue inmovilizado.

4.5.2 Comisiones y valoración

El procedimiento manual aportó reglas de referencia que se han convertido en hipótesis de simulación. BUFF se modela con un factor neto de venta de 0,975. Steam usa una comisión de mercado del 0,87 y, en escenarios conservadores de salida a banco, puede aplicarse una pérdida adicional del 20 %. La conversión inicial simplificada usa $1 \text{ EUR} = 8 \text{ CNY}$, aunque queda pendiente versionar tipos de cambio por fecha si se incorporan fuentes externas.

El *trade hold* se modela de forma conservadora como ocho días desde la compra, equivalente a siete días más una ventana de desbloqueo. Esta simplificación evita declarar vendible una posición demasiado pronto.

Implementación y operación

Este capítulo traslada la propuesta a componentes de software. Describe cómo se organiza el repositorio, cómo se ejecutan los flujos de adquisición y persistencia y cómo la consola local presenta las oportunidades y su trazabilidad.

5.1 Organización del sistema

La implementación se ha organizado como un monorepo para separar el código de dominio, los conectores de adquisición y las herramientas de operación. La separación no responde solo a una preferencia de estructura. Permite ejecutar un scraper, repetir un entrenamiento o consultar el panel sin mezclar esa lógica con las reglas económicas y con los contratos de datos.

Módulo	Función
Dominio y contratos	Tipos, identificadores y validaciones comunes.
Persistencia	Artículos, históricos y señales en PostgreSQL.
Simulación	Cartera, comisiones, bloqueos y riesgo.
Predicción	Variables, entrenamiento e inferencia supervisada.
MARL	Entorno, recompensas y políticas.
Visión	Preprocesado de imagen y OCR.
Aplicaciones	Comandos y consola local.

Tabla 5.1: Módulos del sistema.

Las capas se comunican mediante contratos explícitos. Por ejemplo, un extractor no entrega directamente una recomendación. Entrega una observación con artículo, plataforma, fecha, precio, moneda, volumen, origen y calidad. La persistencia conserva esta información y los módulos de datos construyen después las variables necesarias para cada experimento. Esta secuencia facilita localizar un error sin tener que rehacer todo el flujo.

5.2 Flujo operativo de adquisición

Esta sección explica cómo se ejecuta la recogida de datos y cómo se mantiene su trazabilidad. Se describen las sesiones necesarias para acceder a las fuentes, el proceso de scraping y el almacenamiento incremental de las observaciones.

5.2.1 Sesiones, scraping y trazabilidad

Las fuentes que necesitan navegación se ejecutan con Playwright en un navegador visible cuando es necesario iniciar sesión. La sesión no se guarda en el repositorio ni en el archivo de configuración. Se almacena localmente como estado de navegador y se renueva desde la pantalla de sesiones. Esto evita que las credenciales formen parte de los experimentos o de los commits.

Cada trabajo de scraping registra inicio, conectores utilizados, reintentos, progreso, fichero de log y resultado final. Los límites de concurrencia, el tamaño de lote y el tiempo de espera son configurables. El objetivo no es ejecutar el mayor número de peticiones posible, sino mantener un proceso que pueda detenerse, reanudarse y auditarse si una fuente cambia su interfaz.

Algoritmo 5 Flujo de scraping.

Entrada: Artículos localizados, conectores activos y configuración de trabajo

Salida: Observaciones persistidas y estado de ejecución

- 1: Crear identificador de trabajo y registrar el inicio
 - 2: **for all** lotes de candidatos **do**
 - 3: **for all** conectores habilitados **do**
 - 4: Consultar la fuente con sesión, límite y espera configurados
 - 5: Normalizar precio, moneda, disponibilidad y fecha de observación
 - 6: Validar el resultado y registrar anomalías o reintentos
 - 7: **end for**
 - 8: Persistir observaciones válidas y actualizar el progreso
 - 9: **end for**
 - 10: Registrar resultado, cobertura y referencia del log
-

5.2.2 Persistencia incremental

El modelo operativo almacena el estado actual en `market_items` y las series en `market_history_points`. Guardar el histórico en formato largo permite incorporar nuevas métricas sin alterar el esquema de una tabla ancha. Una observación incluye el origen y la fecha, por lo que una variación de precio puede relacionarse con la fuente y con la ejecución que la recogió.

La deduplicación se realiza antes de persistir y las actualizaciones de estado se comprueban con pruebas de integración. En la validación ejecutada se insertó un snapshot, se actualizó una cotización y se guardó una señal vinculada a un artículo. Cada prueba revierte su transacción al terminar, de modo que las comprobaciones no contaminan el histórico de experimentación.

5.3 Consola de consulta

La consola local convierte el estado técnico en una interfaz de revisión. Su pantalla principal separa la bandeja de oportunidades del detalle de la oportunidad seleccionada. La bandeja concentra el nombre del artículo, la ruta, el margen y los precios Steam y BUFF. El detalle explica qué señal la ha producido, qué datos faltan y qué restricciones de cartera intervienen.

El panel no ejecuta compras. Las acciones de abrir Steam o BUFF sirven para que la persona usuaria revise la oportunidad en su fuente. Los estados *revisar*, *observar* y *bloqueado* representan una decisión explicable, no una orden automática. Esta restricción es importante mientras la validación estadística y multiagente sigue en fase exploratoria.

Vista	Información que hace visible
Oportunidades	Ruta, precios, margen y estado de la señal.
Scraper	Progreso, conectores, logs y resultado.
Sesiones	Vigencia de Steam y BUFF.
Modelo	Versión, conjunto, métricas y carácter experimental.
Agentes y riesgo	Roles, capital bloqueado, límites y rechazos.

Tabla 5.2: Vistas de consola.

5.4 Reproducibilidad y configuración

La configuración se versiona mediante un fichero de ejemplo sin secretos. Para ejecutar la persistencia basta con definir `DATABASE_URL`. El resto de opciones habilita funciones concretas: ruta de Tesseract, token local del servidor de scraping, límites de concurrencia o parámetros de sesiones. Las claves de una base de datos y los estados de navegador quedan excluidos del control de versiones.

Los comandos de construcción de datasets y entrenamiento guardan el rango temporal, las columnas, los tamaños de cada partición, las tasas de clase, los hiperparámetros y las métricas obtenidas. Esta información permite distinguir dos situaciones que visualmente podrían parecer iguales: ejecutar un modelo con más datos y repetir exactamente una configuración anterior.

Algoritmo 6 Ejecución de experimento.

Entrada: Dataset versionado, fechas de corte, configuración y semilla**Salida:** Informe, figura y modelo asociados a una ejecución

- 1: Construir entrenamiento, validación y prueba respetando el orden temporal
 - 2: Guardar el esquema de variables y las tasas de clase
 - 3: Entrenar los candidatos con la configuración declarada
 - 4: Elegir parámetros usando solo validación
 - 5: Evaluar una vez sobre prueba y guardar métricas
 - 6: Generar tabla o figura desde el informe guardado
-

Este procedimiento se aplica también a las pruebas automatizadas. La suite acompaña las modificaciones del sistema, de forma que las tablas y gráficas se apoyen en componentes que han pasado las comprobaciones de dominio, persistencia, OCR, scraping, simulación, MARL y panel web.

Experimentación y análisis

Este capítulo describe las pruebas realizadas sobre reglas económicas, datos, modelos supervisados, OCR y arquitectura multiagente. Cada resultado se interpreta según la evidencia disponible y se distinguen las comprobaciones funcionales de las comparaciones que requieren más datos.

6.1 Diseño experimental

La experimentación se ha desarrollado por fases. Primero, se han migrado reglas económicas y estructuras de datos. Después, se han construido datasets supervisados para estudiar señales de mercado. Más adelante, se ha creado un dataset de trading basado en precios Steam/BUFF y se ha implementado el entorno multiagente. Esta secuencia evita entrenar agentes MARL sobre un entorno que todavía no representa bien el problema económico. La Tabla 6.1 resume las ejecuciones que se desarrollan con detalle en las secciones siguientes.

Flujo	Comprobación ejecutada
Reglas económicas	Pruebas de conversión, comisiones, bloqueo temporal y límites.
Modelo de dirección	Entrenamiento, calibración y perfil de umbrales sobre histórico Steam.
Trading BUFF–Steam	Cortes temporales, ablaciones y comprobación de cobertura.
OCR	Parser, umbral de confianza y recorrido de imagen.
MARL	Dos baselines coordinados y smoke de PPO con tres políticas.

Tabla 6.1: Matriz experimental.

6.1.1 Escenarios de evaluación

Se han usado dos tipos de datasets. El primero predice dirección de mercado sobre histórico disponible y permite estudiar señales temporales de forma amplia. El segundo intenta modelar oportunidades de compraventa Steam/BUFF con precios normalizados, comisiones y horizonte temporal. Este segundo dataset es más cercano al objetivo del TFM, pero también es más exigente porque necesita histórico alineado entre plataformas.

El criterio de evaluación no es solo predictivo. Una señal puede tener buena precisión y aun así no ser operativa si aparece pocas veces, si depende de artículos concretos o

si no supera costes de transacción. Por eso las métricas del modelo supervisado se interpretan como apoyo a la decisión, no como resultado económico final.

6.1.2 Estrategias de comparación

La comparación experimental se plantea en tres niveles. El primero es un baseline simple o dummy para comprobar que los modelos aprenden algo más que la tasa base. El segundo compara modelos supervisados tabulares como regresión logística, random forest e histogram gradient boosting, familias ampliamente usadas para clasificación tabular [8, 9]. El tercero comprueba la arquitectura MARL frente a reglas de margen y de espera. Esta última parte valida coordinación, restricciones y trazabilidad, no rentabilidad de una política entrenada.

Algoritmo 7 Entrenamiento PPO

Entrada: Entorno paralelo, políticas Scout, Trader y Portfolio, semillas y pasos

Salida: Checkpoint y métricas de validación y prueba

- 1: Inicializar una política PPO por rol y las máscaras de acción del entorno
 - 2: **for** cada iteración de entrenamiento **do**
 - 3: Ejecutar episodios y registrar $(o_t^i, a_t^i, R_t, m_t^i)$
 - 4: Estimar ventajas $\hat{A}_t$ a partir de las transiciones registradas
 - 5: Optimizar cada política con el objetivo recortado de PPO
 - 6: **end for**
 - 7: Evaluar el checkpoint en un bloque temporal posterior sin actualizar parámetros
 - 8: Guardar saldo, operaciones, acciones inválidas y recompensa por semilla
-

6.2 Validación técnica y pruebas

Las pruebas no se tratan como material auxiliar, sino como parte de la evidencia experimental del proyecto. Antes de interpretar resultados de modelos o agentes, se comprueba que las piezas que producen esos resultados funcionan de forma aislada: reglas económicas, parsing de mercado, persistencia, construcción de datasets, inferencia supervisada, simulador de cartera, restricciones de riesgo, entorno PettingZoo, adaptador RLlib, comandos de episodios y panel web.

La suite automatizada se organiza principalmente con Pytest y se complementa con Ruff y Mypy. Las pruebas unitarias cubren el comportamiento determinista del simulador, las reglas económicas del procedimiento manual, el OCR, SteamDT, Steam, BUFF, los datasets supervisados, el dataset de trading, el servicio de scoring y los componentes MARL. También existe una prueba de integración para repositorios de persistencia, separada porque depende de una base de datos disponible.

Las pruebas de rendimiento se integran en la evaluación del capítulo porque condicionan la utilidad del sistema. En este proyecto el rendimiento no se mide solo por tiempo de ejecución, sino también por cobertura de datos, número de señales generadas, coste de entrenamiento, capital bloqueado, drawdown y operaciones rechazadas. Los smoke tests de entrenamiento y los episodios MARL registran estas métricas para comprobar que el sistema no solo ejecuta, sino que produce salidas interpretables y comparables entre configuraciones.

Bloque	Qué verifica
Reglas y cartera	Fees, conversión, <i>trade hold</i> , posiciones y límites.
Datos y scraping	Parsing, normalización y construcción de históricos.
Modelo supervisado	Variables, carga de modelo e inferencia.
Simulación MARL	Acciones, observaciones, recompensas y adaptación a RLib.
Interfaz y operación	Estado, recomendaciones y ejecución de tareas.

Tabla 6.2: Cobertura de pruebas.

6.3 Métricas

Las métricas permiten interpretar una estrategia más allá de si produce una señal. Se separan indicadores de retorno y de riesgo para comprobar tanto el posible margen como el uso de capital y las restricciones que condicionan una decisión.

6.3.1 Métricas de rentabilidad

Las métricas de rentabilidad principales son beneficio neto, retorno porcentual, saldo final y número de señales operativas. En los modelos supervisados se usan además precisión, recall, F1, ROC-AUC, PR-AUC y Brier score. Para el uso operativo se da especial importancia a la precisión en umbrales altos, porque una señal menos frecuente pero más fiable puede ser más útil que muchas señales ruidosas. La calibración de probabilidades es relevante porque la salida del modelo se usa como señal de decisión, no solo como etiqueta binaria [10].

6.3.2 Métricas de riesgo

Las métricas de riesgo incluyen drawdown, capital bloqueado medio, posiciones abiertas, exposición por activo o plataforma, operaciones rechazadas por riesgo e impacto del *trade hold*. Estas métricas son necesarias para evaluar si una estrategia es operativamente utilizable y no solo rentable en una métrica aislada.

6.4 Resultados

Esta sección reúne los resultados obtenidos durante la construcción del sistema. Se presentan primero las comprobaciones sobre reglas y modelos y después las pruebas con datos de trading, OCR y agentes cooperativos.

6.4.1 Migración de reglas económicas

La primera fase ha consistido en convertir reglas del procedimiento manual en funciones comprobables. Se han migrado conversiones EUR/CNY, factores de comisión, cálculo de beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado. Esta fase no produce un resultado de rentabilidad por sí misma, pero es necesaria para que los experimentos posteriores midan el mismo dominio que se utilizaba manualmente.

El resultado principal de esta fase es metodológico: las reglas económicas pasan de estar dispersas en fórmulas de hoja de cálculo a estar encapsuladas en módulos de simulación, con pruebas unitarias y parámetros explícitos. Esto reduce errores silenciosos y permite repetir un experimento con la misma configuración.

6.4.2 Resultados del modelo supervisado

La fase supervisada conserva una exploración de dirección Steam con 47.467 filas de entrenamiento, 19.252 de validación y 18.412 de prueba. Su etiqueta indica dirección de precio, no beneficio neto entre plataformas. Se mantiene como evidencia de que el pipeline de variables, calibración y selección funciona con una muestra de mayor tamaño, pero no se utiliza para aprobar compras. El protocolo principal de este capítulo es el corte BUFF–Steam que empieza en diciembre de 2025.

Umbral	Precisión	Recall	Señales
0,50	0,716	0,116	1.293
0,70	0,856	0,019	180
0,80	0,947	0,011	95
0,85	0,962	0,006	53
0,90	1,000	0,003	22

Tabla 6.3: Umbrales de dirección.

El umbral 0,85 deja 53 señales de las 18.412 filas de prueba. Esto ilustra el intercambio entre precisión y cobertura: elevar el umbral selecciona menos casos y no convierte la probabilidad en una orden. La calibración se revisa porque una probabilidad solo es útil si representa de forma razonable la frecuencia observada [10, 11]. Estos resultados no se consideran definitivos. Sí indican que el pipeline de entrenamiento, calibración, selección por umbral y evaluación temporal funciona.

6.4.3 Evolución de los datasets de trading

El dataset de compraventa no apareció en una única ejecución. Se han construido varios cortes porque cada uno reveló una limitación diferente del histórico disponible. El primer `trading_profit_v1` reunió 2.331 filas y 50 artículos. Sirvió para comprobar el constructor de etiquetas, las comisiones y la persistencia de características, pero la dirección analizada solo conservaba un caso positivo en validación y otro en prueba. Por ese motivo sus métricas no se comunican como resultado de un modelo.

Después se corrigieron el parser de BUFF, la normalización de moneda y el backfill de puntos históricos. Con estas correcciones se ejecutaron tres cortes BUFF–Steam desde diciembre de 2025. La Tabla 6.4 deja visibles tanto las muestras que permiten una exploración como las que se descartan. Esta distinción es importante: un dataset grande no es suficiente si sus bloques de validación solo contienen una clase.

La secuencia mejora la cobertura temporal, pero no el equilibrio de etiquetas. El conjunto inicial contiene 50 artículos, pero solo conserva un caso positivo en validación y otro en prueba. En marzo, la proporción de casos positivos es 93,0 % en entrenamiento, 98,7 % en validación y 89,5 % en prueba. Es el único corte que contiene ambas clases en los tres bloques, aunque sigue muy concentrado en oportunidades positivas.

Corte	Entrenamiento	Validación	Prueba
Inicial	2.331	—	—
Marzo	171	76	95
Mayo	361	76	95
Reciente	551	76	84

Tabla 6.4: Cortes de datos.

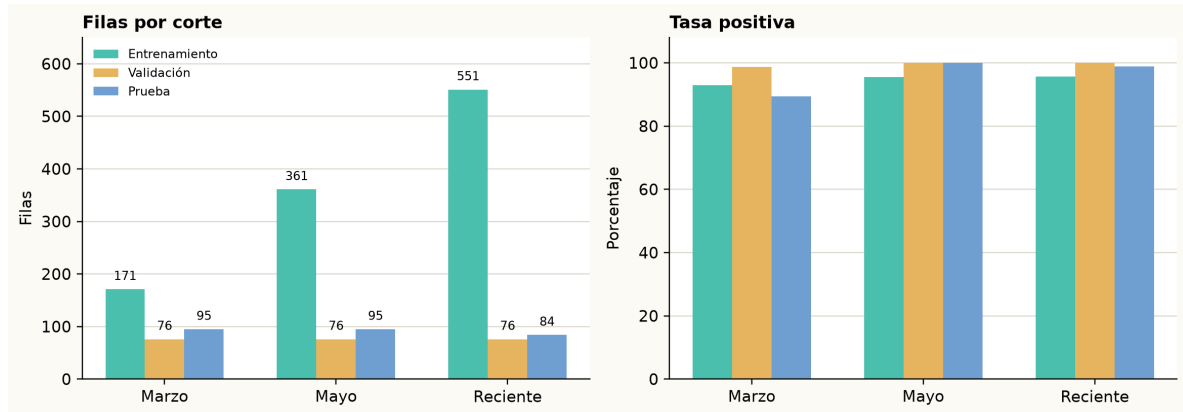


Figura 6.1: Evolución de cortes.

Los cortes de mayo y reciente amplían el número de filas. En mayo, validación y prueba contienen únicamente casos positivos. En el corte reciente las proporciones son 95,6 %, 100 % y 98,8 %, respectivamente. Estas muestras no habilitan ROC-AUC ni calibración sobre los bloques finales, por lo que se conservan como diagnósticos de calidad de datos y no como experimentos de rendimiento.

El flujo reproducible queda definido así: capturar observaciones, normalizar artículo y moneda, construir la etiqueta a ocho días, eliminar la zona de purga, separar por fecha y revisar la distribución antes de entrenar. Solo cuando cada bloque incluye resultados favorables y desfavorables se entrena un clasificador. Esta comprobación previa evitó presentar un entrenamiento trivial como una mejora del sistema.

6.4.4 Protocolo temporal reproducible

La validación se ha repetido el 10 de agosto de 2026 con los datos disponibles desde el 1 de diciembre de 2025. Cada ejemplo representa la posibilidad de comprar un artículo en BUFF y venderlo en Steam tras un horizonte de ocho días. La etiqueta vale uno cuando, después de aplicar las comisiones configuradas, el retorno supera el 10 %. El precio de entrada se toma de BUFF y la salida de Steam se valora con el factor de venta de 0,87 empleado por el simulador.

El orden temporal se conserva en todas las fases. Para el corte de marzo se usaron 171 ejemplos de entrenamiento, 76 de validación y 95 de prueba. El entrenamiento termina el 20 de enero, la validación ocupa del 2 al 20 de febrero y la prueba comprende del 4 al 28 de marzo. Se eliminan ocho días antes de cada frontera para que el horizonte futuro de una observación no alcance el bloque siguiente. Ningún parámetro se modifica

al consultar la prueba. Esta separación evita mezclar pasado y futuro, una precaución esencial cuando las observaciones tienen dependencia temporal [12].

La Figura 6.2 hace visible una limitación que no se aprecia al mirar solo el número de filas. Los tres bloques contienen las mismas 19 representaciones de artículo y están dominados por la clase rentable. La separación temporal evita que se use información futura, pero no demuestra por sí sola que el modelo generalice a artículos que no ha visto durante el entrenamiento.

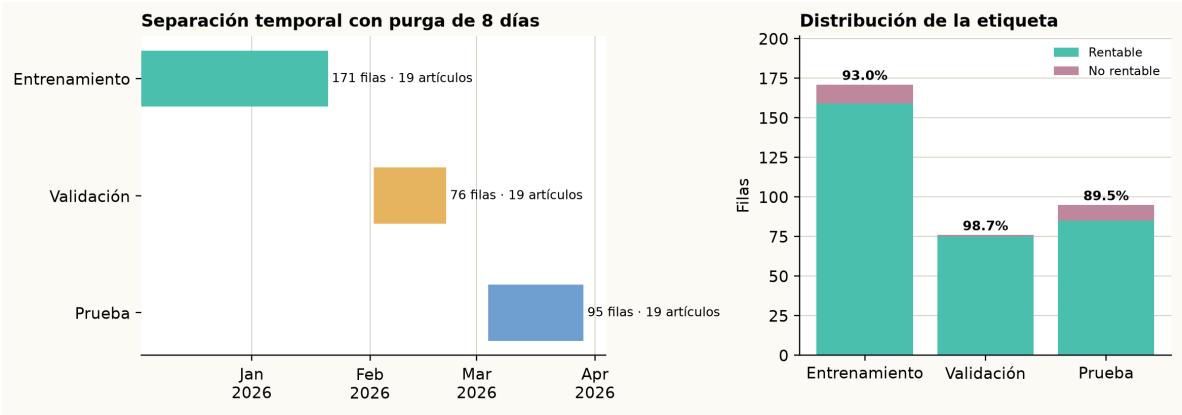


Figura 6.2: Corte temporal de marzo.

La selección compara las familias dummy, regresión logística, random forest e histogram gradient boosting. La configuración se elige en validación mediante precisión al umbral 0,85 con al menos tres señales. Después se calibra con sigmoide y se mide sobre el bloque de prueba. Además, se ejecuta una ablación de la regresión logística sin retardos, cambios, retornos ni medias móviles de uno, tres y siete días. Esta comparación responde a una pregunta concreta: si el histórico añade información o solamente complejidad. La precisión se presenta junto a la cobertura porque una clase mayoritaria puede hacer que el ROC-AUC parezca más concluyente de lo que permite la muestra [13].

La exploración no se limitó a escoger una familia. La Tabla 6.5 recoge el espacio de búsqueda empleado. El conjunto de prueba se dejó fuera de esta selección: sus valores se consultaron una vez después de fijar la configuración de validación.

Familia	Configuraciones exploradas
Regresión logística	Regularización L2 con $C \in \{0,3,1,3\}$.
Random forest	Profundidad 10 o 18, 160 árboles y cinco muestras por hoja.
HGB	Tasa de aprendizaje 0,05 o 0,10, 160 iteraciones y 31 hojas.

Tabla 6.5: Configuraciones de marzo.

La exploración inicial señala que la regresión logística sin características históricas puede separar mejor las dos clases que las alternativas probadas. También muestra que los retardos y medias móviles no deben darse por útiles sin contrastarlos. La siguiente ablación vuelve a medir esa decisión junto con la aumentación, usando particiones por fecha para que una fecha completa no se divida entre lados de un pliegue.

Configuración	ROC-AUC	Brier	Precisión @0,85	Señales
Logística con histórico	0,641	0,075	0,929	85
Logística sin histórico	0,760	0,072	0,970	67
Random forest con histórico	0,742	0,074	0,942	86
HGB con histórico	0,649	0,082	0,915	82

Tabla 6.6: Comparativa de marzo.

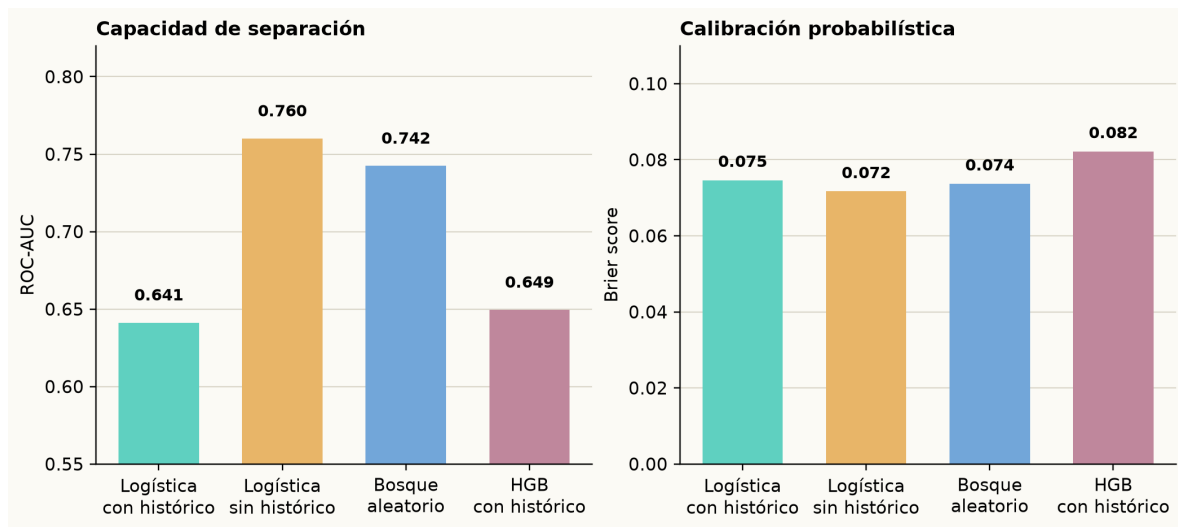


Figura 6.3: Modelos en marzo.

6.4.5 Ablación de histórico y aumentación

La segunda exploración usa una matriz 2×2 con regresión logística. Se mantienen los mismos cortes temporales de marzo, el umbral de selección 0,85, tres pliegues temporales y calibración sigmoide. Se comparan dos decisiones de diseño: conservar o retirar retardos y medias móviles, y entrenar con o sin aumentación numérica. La aumentación se contrasta como hipótesis y no se presupone beneficiosa, ya que su efecto depende del dominio y del volumen de datos [14].

La aumentación se aplica únicamente al entrenamiento. Para cada variable numérica se genera una copia perturbada mediante

$$x'_k = x_k + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, 0,01 \cdot IQR(x_k)).$$

Las etiquetas, las variables categóricas y las fechas no cambian. Además, todas las filas de una misma fecha se mantienen dentro del mismo lado de cada pliegue temporal. De este modo la copia perturbada no puede aparecer en entrenamiento mientras su original está en validación. La validación y la prueba permanecen sin modificar.

La combinación sin histórico ni aumentación obtiene el mejor equilibrio entre separación y calibración. El jitter duplica el tamaño de entrenamiento de 171 a 342 filas, pero empeora ROC-AUC y Brier en ambas configuraciones. Por ello no se incorpora al modelo de referencia. Este resultado no prueba que la aumentación sea perjudicial en cualquier muestra. Indica que, con una colección corta y muy concentrada en

Histórico	Aumentación	ROC-AUC	Brier	Precisión @0,85	Señales
Sí	No	0,631	0,076	0,931	87
No	No	0,716	0,076	0,952	62
Sí	Jitter	0,575	0,085	0,924	92
No	Jitter	0,613	0,084	0,919	86

Tabla 6.7: Ablación logística.



Figura 6.4: Histórico y aumentación.

operaciones positivas, introducir pequeñas variaciones de precio añade más ruido que información.

La lectura debe mantenerse prudente. Los 19 artículos disponibles en el corte aparecen tanto en entrenamiento como en prueba y la etiqueta positiva representa el 89,5 % de la prueba. Una estrategia que marca muchos casos como favorables puede alcanzar precisión elevada por la propia distribución de la muestra. El corte de mayo confirma este problema: de 361 ejemplos de entrenamiento, 76 de validación y 95 de prueba, las dos últimas particiones contienen exclusivamente etiquetas positivas. En esas condiciones ROC-AUC no es una métrica definida y no se entrena un modelo para presentarlo como comparación válida. Este resultado establece una condición de continuación clara: ampliar la cobertura cruzada BUFF–Steam y equilibrar los eventos antes de convertir el modelo en recomendación operativa.

La Figura 6.5 complementa esta lectura. La precisión se mantiene cerca de la tasa positiva de la prueba mientras el umbral aumenta, pero el número de señales solo cae de forma apreciable a partir de 0,90. Por tanto, un umbral alto no convierte por sí mismo la señal en evidencia de selección robusta. La calibración y la precisión deben volver a evaluarse cuando haya una proporción relevante de operaciones no rentables.

6.4.6 Comprobación del corte reciente

Con la base de datos conectada se repitió la construcción del dataset de BUFF a Steam el 11 de agosto de 2026. Se empleó la ventana desde diciembre de 2025, un horizonte de ocho días, una purga de ocho días y una prueba acotada a julio de 2026. La comprobación conserva los mismos 19 artículos. El resultado no habilita otro entrena-

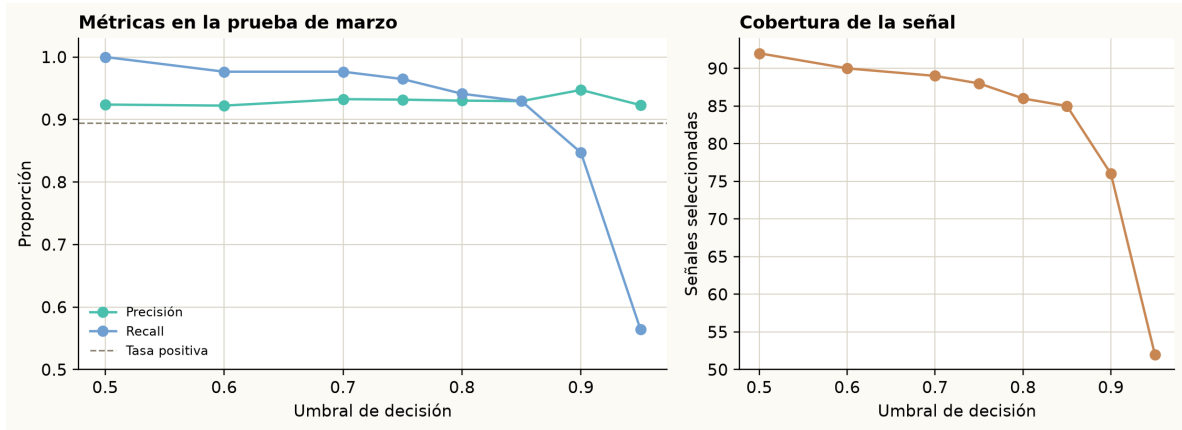


Figura 6.5: Métricas por umbral.

miento. Sirve para comprobar si la nueva captura ha aportado variedad suficiente de etiquetas.

Entrenamiento	Validación	Prueba
551 (95,6 %)	76 (100 %)	84 (98,8 %)

Tabla 6.8: Corte BUFF–Steam.

El resultado es deliberadamente incómodo, pero es importante. En validación y prueba falta una clase negativa, por lo que ROC-AUC, calibración y una comparación de políticas no son interpretables. Entrenar con esta muestra produciría una clasificación trivial y no aportaría evidencia sobre el sistema. La prioridad experimental pasa a ser mantener capturas recurrentes de más artículos, incluir oportunidades descartadas y conservar decisiones cerradas para que la recompensa MARL pueda distinguir compras acertadas, compras fallidas y decisiones de espera.

6.4.7 Evidencia de pruebas automatizadas

La suite completa se ejecutó el 11 de agosto de 2026 sobre el repositorio restaurado y obtuvo 278 pruebas superadas en 35,20 segundos. Las tres pruebas de integración utilizaron el DATABASE_URL PostgreSQL configurado en Supabase. Registran un snapshot de mercado, comprueban la actualización de una cotización y guardan una señal de oportunidad vinculada a un artículo. Cada una abre una transacción externa y la revierte al terminar, por lo que la comprobación no añade datos de prueba al histórico utilizado por los experimentos. Este registro de datos, configuración y resultados permite repetir las comprobaciones, práctica recomendada para hacer evaluables los resultados de aprendizaje automático [15].

6.4.8 Validación funcional de OCR

La evaluación de OCR disponible es funcional, no una medida de exactitud sobre un corpus etiquetado de capturas. Por ello no se comunica una tasa de reconocimiento que no se ha medido. Las pruebas comprueban que el parser convierte una observación de mercado en un contrato con artículo, calidad, plataforma, precio, moneda y volumen,

Bloque	Pruebas	Alcance
Unitarias actuales	275	Economía, OCR, conectores, scraping, datasets, simulador, MARL y panel web.
Integración	3	Persistencia de snapshots, actualización de precios y señales en PostgreSQL.
Suite completa	278	Todas superadas en 35,20 segundos.

Tabla 6.9: Pruebas automatizadas.

que rechaza una confianza inferior a 0,5 y que el recorrido de imagen llama a carga, preprocesado y reconocimiento antes de persistir el resultado. El uso del motor se apoya en la arquitectura de Tesseract descrita por Smith [16]. La Tabla 6.10 delimita qué evidencia existe y qué queda pendiente.

Prueba	Caso ejecutado
Fixture de texto	Texto Steam y BUFF convertido a observaciones.
Umbral de confianza	Confianza 0,2 con mínimo configurado en 0,5.
Recorrido de imagen	Carga, preprocesado y reconocimiento con un runner controlado.

Tabla 6.10: Validación funcional de OCR.

La fixture confirma que el parser construye observaciones con los campos esperados a partir de texto de mercado. La prueba de confianza rechaza el valor 0,2 al exigir un mínimo de 0,5. El recorrido de imagen completa la secuencia de carga, preprocesado y reconocimiento y devuelve una confianza de 0,91 en el caso preparado.

Estas comprobaciones validan la integración, no la exactitud del OCR sobre capturas de interfaz. Por eso el siguiente experimento debe usar un conjunto etiquetado y comunicar por separado la exactitud de artículo, calidad, plataforma, precio y moneda. Hasta disponer de ese conjunto, el OCR se presenta como un conector validado y no como un sistema de reconocimiento cuantificado.

6.4.9 Dataset de trading operativo

El dataset `trading_profit_v1` se ha generado desde `market_history_points` usando precios Steam/BUFF, comisiones y horizonte temporal. La primera versión ha producido 2.331 ejemplos y 50 artículos, pero con desbalance extremo en validación y test: solo un caso positivo en cada split para la dirección analizada. Por este motivo, las métricas de ese entrenamiento no se usan como evidencia de rendimiento operativo.

El análisis posterior ha mostrado que el histórico de BUFF ha empezado a estar mejor alineado tras corregir el parser y permitir backfill de puntos históricos. Aun así, la dirección más natural de arbitraje, comprar en BUFF y vender en Steam, todavía no genera ejemplos suficientes a ocho días porque falta histórico futuro Steam alineado con los puntos BUFF recientes.

A partir de esta limitación, la validación distingue entre aprendizaje temporal e inferencia operativa: Steam se usa como fuente principal para aprender el riesgo temporal

de salida y BUFF queda como precio vivo de entrada en inferencia. El objetivo no es predecir el precio exacto de BUFF. La recomendación evalúa si un precio BUFF puntual conserva margen suficiente frente a una salida Steam estimada y sus comisiones.

6.4.10 Dificultades y pivotaciones

Durante el desarrollo se han producido varias decisiones de pivotaje. La primera aparece al formalizar el procedimiento manual: el valor principal estaba en extraer reglas económicas verificables y convertirlas en simulación reproducible. Esta decisión desplaza el trabajo desde una herramienta manual hacia un sistema auditable.

La segunda pivotación afecta a las fuentes de datos. El planteamiento inicial intentaba alinear históricos de Steam y BUFF para entrenar directamente sobre oportunidades de arbitraje entre plataformas. Las primeras pruebas mostraron dos problemas: falta de histórico alineado suficiente y riesgo operativo al consultar BUFF de forma reiterada. Por ello, el enfoque se ajusta: Steam se mantiene como fuente principal de aprendizaje temporal y BUFF se usa de forma puntual como precio vivo de entrada para evaluar flips concretos.

La tercera pivotación aparece en la parte multiagente. Antes de ampliar entrenamiento y episodios, se prioriza que el entorno, las recompensas, las restricciones y las métricas sean comprobables. Esta secuencia permite construir el entrenamiento sobre reglas económicas trazables, datos consistentes y comparaciones con baselines.

6.4.11 Protocolo de episodios MARL

La evaluación multiagente no utiliza una lista abstracta de acciones. Cada episodio recorre las observaciones del bloque de prueba de marzo en orden temporal. Scout indica si una oportunidad merece atención, Trader propone comprar una unidad o mantener y Portfolio aplica las restricciones de cartera. Solo se abre una posición si los tres roles son compatibles. De este modo, el episodio permite comprobar el intercambio entre una oportunidad local y el capital disponible para el conjunto de la cartera.

Elemento	Configuración comprobada
Observaciones	95 filas del bloque de prueba de marzo, en orden temporal.
Roles	Scout, Trader y Portfolio con observaciones locales.
Capital inicial	1.000 EUR en los baselines de margen y mantener.
Acción de compra	Requiere acuerdo de los tres roles y límites de cartera.
Estado de cartera	Efectivo, posiciones, exposición, bloqueo y drawdown.
Recompensa	Margen estimado menos inactividad, restricciones, drawdown y capital bloqueado.
Smoke PPO	Una iteración y 64 pasos mediante el adaptador PettingZoo-RLlib.

Tabla 6.11: Episodio MARL.

La diferencia entre los baselines y PPO debe mantenerse clara. Las reglas de margen y de mantener se usan para verificar que el entorno reacciona de la forma esperada. El

smoke de PPO comprueba que el mismo entorno puede entregar observaciones, recompensas y máscaras de acción a RLLib. Ninguno de estos dos pasos constituye todavía una comparación de políticas entrenadas.

6.4.12 Resultados MARL

El entorno multiagente se ha comprobado con el mismo dataset de marzo. Scout marca una oportunidad, Trader propone la compra de una unidad y Portfolio acepta o rechaza según saldo, exposición, liquidez y capital bloqueado. Una compra requiere el acuerdo de los tres roles. La recompensa compartida combina margen, inactividad, penalización por restricciones, drawdown y capital inmovilizado:

$$r_t = w_m \rho_t \mathbb{I}_{\text{compra}} - \lambda_i \mathbb{I}_{\text{oportunidad ignorada}} - \lambda_r v_t - \lambda_d DD_t - \lambda_b B_t.$$

En la ecuación, ρ_t es el retorno estimado de la oportunidad, v_t el número de restricciones incumplidas, DD_t el drawdown y B_t la proporción de capital bloqueado. La misma recompensa se entrega a los tres agentes para favorecer coordinación en lugar de optimización aislada.

Escenario	Política evaluada	Pasos
Margen positivo	Regla de compra por margen con límites de cartera.	95
Mantener	Regla sin apertura de posiciones.	95
Margen sin señal	Regla de margen sin probabilidad supervisada por fila.	95
PPO, una iteración	PPO multiagente mediante PettingZoo-RLLib.	64

Tabla 6.12: Escenarios MARL.

La regla de margen positivo abre 13 posiciones en los 95 pasos y reduce el efectivo disponible hasta 309,64 EUR. La política de mantener no abre posiciones. En la variante sin señal se repiten las 13 aperturas porque el dataset todavía no aporta una probabilidad supervisada por fila. Estos resultados muestran coordinación entre los tres roles y aplicación de las restricciones de Portfolio, no una política aprendida.

El smoke de PPO completa 64 pasos y guarda un *checkpoint*, pero no abre compras en esa única iteración. En los tres escenarios con reglas se partió de 1.000 EUR y se recorrieron las 95 observaciones del bloque de prueba. Las posiciones quedan abiertas por el bloqueo temporal, por lo que los escenarios muestran transiciones del entorno y no beneficio final.

La Figura 6.6 muestra la traza completa de ambos baselines. La regla de margen positivo abre trece posiciones y reduce el efectivo disponible hasta 309,64 EUR. El capital queda bloqueado durante el periodo de espera y deja de contabilizarse como bloqueado al vencer dicho periodo. Las posiciones siguen abiertas porque el baseline no incorpora una política de venta. Esta figura valida la dinámica de efectivo, bloqueo y restricciones del simulador, pero no mide el rendimiento de una política aprendida.

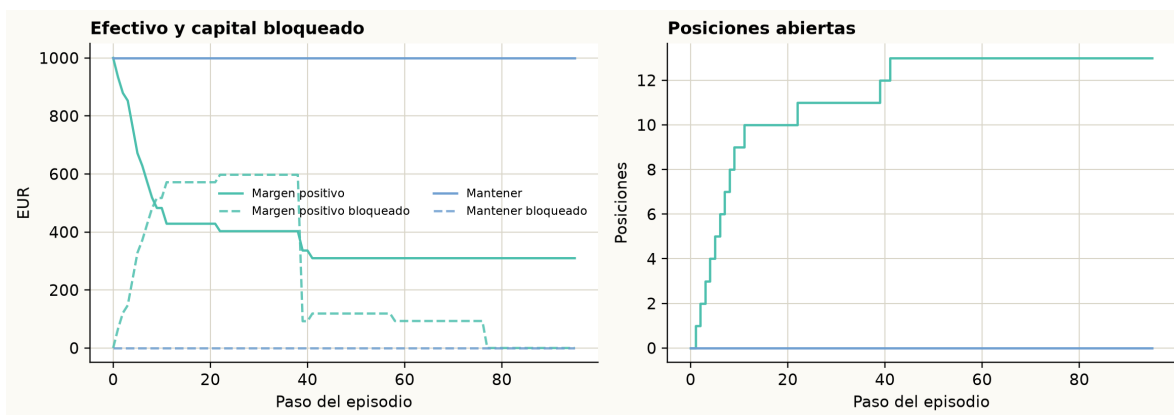


Figura 6.6: Trazas MARL.

El comando PPO con RLlib completó una iteración y 64 pasos de entorno con el conjunto de entrenamiento. El estado central se expone al adaptador y hay una política por rol. La evaluación del checkpoint corto no ejecutó compras. Esta prueba valida la compatibilidad técnica, pero una única iteración no permite comparar una política aprendida con una regla de margen. Las comparaciones futuras deberán repetir semillas, episodios y cortes temporales para comunicar la variabilidad del aprendizaje [17].

6.5 Experimentos finales pendientes

Para cerrar la validación del TFM será necesario ejecutar una fase experimental más estricta. Primero debe validarse el modelo Steam-only de salida a ocho días y conectarlo a candidatos con precio BUFF vivo. Después deben compararse baselines simples: comprar todos los candidatos de SteamDT, comprar solo cuando el margen BUFF $\rightarrow$ Steam es positivo, agente único y política dummy.

La evaluación MARL deberá incluir ablations. Las más relevantes son: sin probabilidad supervisada, sin agente Portfolio, sin penalización por capital bloqueado y sin restricciones de liquidez. Estas comparaciones permitirán distinguir si la mejora viene de la arquitectura multiagente, del modelo supervisado, de una regla de riesgo o simplemente de la estructura del simulador.

6.6 Discusión

La discusión relaciona los resultados con el alcance del trabajo. Se identifican las condiciones que limitan su interpretación, las partes que ya funcionan de forma integrada y los datos que faltan para una evaluación más sólida.

6.6.1 Amenazas a la validez

La principal amenaza interna es el desbalance de etiquetas: la prueba de marzo contiene 85 casos rentables de 95 y los bloques posteriores llegan a tener una sola clase. La segunda es el solapamiento de artículos entre entrenamiento y prueba. La división por fecha protege frente al uso de futuro, pero no responde a la pregunta más exigente de generalizar a variantes nuevas. La tercera amenaza es operativa: el precio BUFF se

usa como entrada puntual y la salida Steam se estima con histórico y comisiones, por lo que la sincronización exacta de ambos mercados requiere más capturas.

También hay límites de medición. El OCR está validado a nivel funcional, pero aún no dispone de un corpus de imágenes etiquetado. Los baselines del entorno MARL verifican transiciones y restricciones, aunque no incluyen una regla de venta completa ni una política entrenada durante suficientes episodios. Estas limitaciones no invalidan la arquitectura, pero acotan las conclusiones a viabilidad técnica y experimental preliminar.

6.6.2 Casos favorables

El trabajo realizado muestra que el sistema ya puede integrar fuentes, construir datasets, entrenar modelos supervisados, calibrar probabilidades, simular reglas económicas y ejecutar un smoke multiagente. La separación entre adquisición, predicción, simulación y decisión ha resultado útil porque permite avanzar por partes y detectar bloqueos concretos sin rehacer toda la arquitectura.

6.6.3 Limitaciones observadas

La principal limitación actual es la disponibilidad de histórico alineado entre Steam y BUFF. Sin suficientes ejemplos positivos en validación y test, cualquier métrica de trading puede ser engañosa. También existe riesgo de sobreajuste en variables categóricas de alta cardinalidad y en señales de precisión alta con pocos casos. Por último, el entorno MARL necesita una validación experimental más amplia antes de poder concluir si la arquitectura multiagente mejora a una estrategia simple.

6.6.4 Lectura de los resultados

Los resultados confirman la viabilidad técnica de la integración y la simulación. La comparación del rendimiento económico entre políticas multiagente requiere una fase experimental adicional con más episodios, datos y baselines homogéneos.

Conclusiones

Este capítulo resume las aportaciones del trabajo, revisa el grado de cumplimiento de los objetivos y delimita los pasos necesarios para ampliar la validación experimental y el entrenamiento multiagente.

7.1 Conclusiones principales

El desarrollo realizado hasta el momento permite concluir que el problema no puede reducirse a una predicción aislada de subida o bajada de precio. En mercados como el de *Counter-Strike 2*, la decisión depende de fricciones muy concretas: comisiones, conversión de moneda, liquidez, *trade hold*, capital bloqueado y riesgo de cartera. Por ello, la arquitectura propuesta necesita combinar adquisición de datos, simulación económica y decisión multiagente, siguiendo la disciplina de backtesting y control de fricciones propia de los entornos de trading con aprendizaje automático [2].

También se ha comprobado que la separación por capas facilita el avance del proyecto. Los agentes extractores se encargan de observar el mercado, el modelo supervisado produce señales probabilísticas, el simulador aplica reglas económicas y los agentes MARL operan dentro de un entorno controlado. Esta división reduce el acoplamiento y permite validar cada parte por separado.

La contribución principal de esta fase no es afirmar que el sistema gane dinero, sino construir una base reproducible para estudiar esa pregunta. El proyecto ya dispone de una arquitectura modular, un flujo de adquisición, un modelo de persistencia, un simulador económico, datasets derivados, modelos supervisados preliminares y un entorno multiagente funcional para smoke tests.

Una conclusión metodológica relevante es que el proyecto ha evolucionado a partir de las dificultades encontradas. La falta de histórico Steam/BUFF suficientemente alineado, el riesgo de scraping reiterado y el desbalance de los primeros datasets de trading han llevado a separar entrenamiento e inferencia operativa. Esta pivotación no cambia el objetivo del sistema, pero sí mejora su viabilidad: el aprendizaje temporal se apoya en Steam y la comparación con BUFF se reserva para evaluar oportunidades concretas.

7.2 Cumplimiento de objetivos

Se ha avanzado en la mayoría de objetivos técnicos: existe un monorepo modular, un modelo de datos operativo, pipelines de adquisición, análisis del procedimiento manual inicial, construcción de datasets, entrenamiento supervisado, simulador de cartera y un entorno MARL funcional para pruebas cortas. También se han definido agentes Scout, Trader y Portfolio, junto con restricciones de riesgo y métricas de evaluación.

El cumplimiento experimental todavía es parcial. Los resultados supervisados muestran que el pipeline funciona y que algunos modelos producen señales de alta precisión en umbrales exigentes, pero el número de señales es limitado. El dataset operativo de trading Steam/BUFF está construido, aunque su primera versión presenta desbalance extremo en validación y test. El entorno MARL ejecuta, pero no constituye todavía una evaluación comparativa completa. Las pruebas automatizadas y los smoke tests de rendimiento forman parte del capítulo experimental porque verifican que las métricas obtenidas proceden de componentes comprobados.

7.3 Limitaciones

La limitación más importante es la escasez de histórico alineado entre plataformas para algunas direcciones de trading. El dataset más cercano al objetivo operativo presenta pocos positivos en validación y test, por lo que no permite defender todavía una mejora robusta. Además, algunos resultados supervisados tienen alta precisión en umbrales altos, pero con pocas señales, lo que obliga a interpretarlos con prudencia. Por ello, se separa el aprendizaje temporal basado en Steam del uso puntual de BUFF como precio vivo de entrada para flips.

Otra limitación es que el smoke multiagente demuestra que el flujo ejecuta, pero no que el sistema haya aprendido una política superior. Para llegar a esa conclusión se necesita una evaluación reproducible con más episodios, más datos, baselines justos y métricas de riesgo.

También existe una limitación propia del dominio. Los activos de CS2 dependen de plataformas externas, reglas de intercambio, sesiones, cambios visuales y restricciones que pueden variar. Por tanto, cualquier sistema automatizado debe mantener trazabilidad, parámetros versionados y capacidad de revisar errores de fuente.

7.4 Trabajo futuro

Las líneas inmediatas de trabajo son validar un modelo Steam-only de salida a ocho días, usar BUFF como precio vivo de entrada en inferencia de flip, comparar contra baselines simples y entrenar MARL con una configuración CTDE completa. Después será necesario estudiar ablations, generar gráficas de resultados y cerrar una discusión experimental sobre cuándo la arquitectura multiagente aporta valor y cuándo no.

Los experimentos finales deberán reportar beneficio neto, saldo final, drawdown, capital bloqueado, número de operaciones, posiciones abiertas y rechazos por riesgo. Además, deberán comparar la arquitectura completa contra versiones simplificadas: sin probabilidad supervisada, sin agente Portfolio y con reglas simples de spread. Solo con esa comparación será posible defender si la descentralización multiagente aporta una mejora frente a estrategias más sencillas, siguiendo la lógica comparativa habitual en MARL y entrenamiento cooperativo [5, 6].

Bibliografía

- [1] Vili Lehdonvirta. Virtual item sales as a revenue model: Identifying attributes that drive purchase decisions. *Electronic Commerce Research*, 9(1):97–113, 2009.
- [2] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance*, 2021.
- [3] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [5] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [6] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. In *Advances in Neural Information Processing Systems*, 2022.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>, 2017.
- [8] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [9] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [10] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632, 2005.
- [11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017.
- [12] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018.
- [13] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

- [14] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [15] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.
- [16] Ray Smith. An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition*, volume 2, pages 629–633, 2007.
- [17] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.
- [18] Justin K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Diefendahl, Niall L. Williams, Yashas Lokesh, Praveen Ravi Hines, and Niall Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.
- [19] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

Anexos

Los anexos reúnen detalles operativos que complementan la explicación principal: estructura del repositorio, configuración del entorno, comandos reproducibles y parámetros de entrenamiento.

A.1 Estructura del repositorio

El repositorio se organiza como un monorepo. La carpeta **apps** contiene aplicaciones y comandos ejecutables. La carpeta **packages** contiene la lógica reutilizable. La carpeta **docs** recoge decisiones y documentación técnica. La carpeta **tests** contiene pruebas unitarias e integración. La carpeta **TFM** mantiene la memoria en L^AT_EX.

Los módulos principales son:

- **apps/acquisition**: extractores, scraping, OCR, importadores y SteamDT.
- **apps/cli**: comandos de construcción de datasets, entrenamiento, scraping y evaluación.
- **apps/web**: interfaz web local de consulta y control operativo.
- **packages/contracts**: contratos Pydantic compartidos.
- **packages/persistence**: conexión, repositorios y persistencia.
- **packages/prediction**: entrenamiento e inferencia supervisada.
- **packages/simulation**: reglas económicas, cartera, riesgo y backtesting.
- **packages/marl**: entorno multiagente, wrappers PettingZoo/RLlib y entrenamiento.
- **packages/vision**: OCR, preprocesado y parsing visual.

A.2 Configuración del entorno

El entorno se define en `pyproject.toml`. El proyecto usa Python, Supabase/Postgres, Pytest, Ruff, Mypy y dependencias opcionales para MARL como Ray/RLlib, PettingZoo, Gymnasium y PyTorch. Para desarrollo local existe un `docker-compose.yml` con Postgres.

Comandos representativos:

```
python -m pytest tests/unit
python -m ruff check
python -m mypy apps packages tests/unit
python -m apps.cli.auto_scrape_loop --once
python -m apps.cli.render_stream_scrape
python -m apps.cli.web_dashboard_server
```

A.3 Comandos de datasets y modelos

Los comandos de datos y modelos separan adquisición, construcción de dataset, entrenamiento e inferencia. Esta separación permite repetir una fase sin rehacer todo el flujo.

```
python -m apps.cli.build_supervised_dataset
python -m apps.cli.analyze_supervised_coverage
python -m apps.cli.train_supervised_model
python -m apps.cli.build_trading_dataset
python -m apps.cli.retrain_trading_model
python -m apps.cli.score_live_opportunities
```

A.4 Parámetros de entrenamiento

Los experimentos supervisados usan splits temporales, calibración de probabilidades y selección por precisión operativa en umbrales altos. En los smoke tests se han comparado modelos dummy, regresión logística, random forest e histogram gradient boosting. El entrenamiento MARL usa RLlib con un entorno multiagente y tres políticas: Scout, Trader y Portfolio.

Bloque	Parámetros relevantes
Supervisado	Split temporal, umbral operativo, calibración, modelo candidato y columnas excluidas por leakage.
Trading	Dirección de arbitraje, horizonte, fees, tipo de cambio, ventana histórica y target de beneficio neto.
Simulación	Saldo inicial, tamaño máximo de posición, <i>trade hold</i> , comisiones y límites de exposición.
MARL	Número de episodios, políticas Scout/Trader/Portfolio, algoritmo PPO y métricas de cartera.

Tabla A.1: Configuración experimental.

A.5 Modelo de datos

El modelo operativo actual gira alrededor de dos tablas principales: `market_items` y `market_history_points`. La primera representa variantes concretas de artículos y su estado actual por plataforma. La segunda almacena series temporales en formato largo, con una fila por artículo, plataforma, fecha y métrica. Esta estructura evita mezclar métricas distintas y permite añadir nuevas señales sin alterar la forma general del histórico.

El esquema canónico previsto añade entidades como `assets`, `platforms`, `market_observations`, `predictions`, `agent_actions`, `investment_decisions` y `simulated_positions`. El objetivo de este esquema es mejorar trazabilidad, auditoría y separación entre observación, predicción y decisión.

A.6 Resultados adicionales

Los resultados intermedios se guardan en `model-runs` y los datasets derivados en `data/datasets`. Estas carpetas permiten conservar reportes de cobertura, exploración de características, métricas de entrenamiento, umbrales seleccionados y salidas de smoke tests sin mezclar los artefactos pesados con el código principal.

Para que un experimento aporte evidencia reproducible, debe registrar como mínimo: configuración usada, periodo temporal, número de activos, tamaño de cada split, tasa de positivos, modelo o política evaluada, métricas principales y limitaciones observadas.